Backend per a aplicacions web

PID_00239624

Daniel Julià Lundgren

Temps mínim de dedicació recomanat: 5 hores



Universitat Oberta de Catalunya

Cap part d'aquesta publicació, incloent-hi el disseny general i la coberta, no pot ser copiada, reproduïda, emmagatzemada o transmesa de cap manera ni per cap mitjà, tant si és elèctric com químic, mecànic, òptic, de gravació, de fotocòpia o per altres mètodes, sense l'autorització prèvia per escrit dels titulars del copyright.

Índex

Introducció..... 5

1. WordPress com a *framework* de desenvolupament web..... 7
1.1. Definició de tipus de continguts, camps personalitzats i taxonomies 8
1.1.1. Definició de tipus de contingut 9
1.1.2. Definició de camps personalitzats 10
1.1.3. Definició de taxonomies 12
1.1.4. Mides d'imatge personalitzades 12
1.1.5. Exemple complet 13
1.2. Definició de nous rols i permisos 16
1.3. CMS *headless* i *decoupled*..... 17
1.4. Les API de WordPress 19
1.4.1. Mòdul de gestió d'usuaris 19
1.4.2. Gestió d'arxius (*file management*) 19
1.4.3. Sistema de plantilles (*template management*) 20
1.4.4. Sistema de base de dades (*database management*) 20
1.4.5. Rutes (*Routing*) 20
1.4.6. XML-RPC 20
1.4.7. Programació de tasques (*Scheduling*) 20
1.4.8. *Plugins*..... 21
1.4.9. Configuració de l'escriptori 21
1.5. *Hooks*, filtres i accions 21
1.6. *Widgets*..... 22
1.7. *Shortcodes*..... 24
1.8. Desenvolupament de *plugins* amb WordPress 25
1.9. API REST WordPress 27

2. Exemple d'aplicació..... 29

3. Desenvolupament d'aplicacions web en Drupal..... 31
3.1. Drupal com a *framework*..... 31
3.2. Millores de Drupal 8 respecte a la versió 7 31
3.3. Drupal 8 enfocat a desenvolupament *backend*..... 32
3.4. Drupal 8 i Symfony 32
3.5. Sistema de temes Twig 33
3.6. Classy 34
3.7. Desenvolupament de mòduls a Drupal 35
3.8. Drupal *headless*..... 35
3.9. API Rest Drupal 36

4. *Frameworks* MVC 38
4.1. Funcionalitats d'un MVC 39
4.2. CodeIgniter 40
4.2.1. Exemple CodeIgniter 41
4.2.2. Bases de dades a CI 46
4.2.3. Extensions de CI 46
4.3. Laravel 48
4.4. Symfony 48
4.5. JavaScript al servidor, MEAN (MongoDB, Express, Angular, NodeJS) 48
4.5.1. MEAN 48
4.5.2. MongoDB 48
4.5.3. Node.js 49
4.5.4. Npm 50
4.5.5. Express 51
4.5.6. Loopback 52
4.6. Xarxes socials de codi 52
4.7. Gestors de paquets 53

5. Estudis de cas 55
5.1. Cas 1. Catàleg educatiu interactiu 55
5.1.1. Definició dels requeriments de l'aplicació 55
5.1.2. Anàlisi de pros i contres per a la solució en WordPress, Drupal i CodeIgniter 56
5.2. Cas 2. Aplicació mòbil 57

Introducció

Una aplicació web és una aplicació de programari client-servidor en la qual la interfície d'usuari (la part del client) s'executa a un navegador web.

Els darrers anys el World Wide Web ha evolucionat des d'un sistema de publicació de continguts multimèdia (text, imatge, vídeo) amb enllaços fins a convertir-se en una plataforma de programació potent gràcies a les noves aportacions de l'HTML5 i els llenguatges de programació web.

El desenvolupament d'aplicacions web requereix l'ús d'eines que van molt més enllà de l'HTML, el CSS i el JavaScript. Aquestes eines poden ser en forma de CMS (*content management system*, o sistema de gestió de continguts) o *frameworks* de programació i tota mena de metodologies i eines auxiliars associades.

Aquesta evolució ha estat facilitada en part per l'espectacular millora de l'HTML5 i el JavaScript als navegadors els darrers anys.

Les millores es tradueixen per una banda al CSS, per exemple proporcionant animacions 3D que aprofiten la targeta gràfica de l'ordinador sense haver d'escriure una línia de codi en JavaScript, millores en HTML5 que permeten per exemple incloure vídeo o àudio fàcilment, o millores en les API de JavaScript, algunes d'elles encaminades a facilitar el desenvolupament d'aplicacions mòbils.

El territori del web s'ha expandit molt més enllà del web d'escriptori, estenent-se als dispositius mòbils i tauletes, o fins i tot objectes interconnectats que interactuen amb una API al web. Per exemple, és possible desenvolupar aplicacions per a mòbils fent servir tecnologies web (HTML/CSS/JavaScript) amb llibreries com Cordova o PhoneGap.

En aquesta assignatura ens centrarem en la part del desenvolupament del *backend* de les aplicacions web. Per aquest motiu aprofundirem en CMS com Wordpress o Drupal i veurem com fer-los servir com una plataforma de desenvolupament d'aplicacions.

També farem una breu introducció a *frameworks* de programació MVC, com CodeIgniter o Express, eines més complexes però que permeten una gran flexibilitat.

Per ampliar informació

HTML5 Rocks. Recursos i articles sobre HTML5, CSS i JavaScript (http://www.HTML5rocks.com/en/).

Can I Use. Directori de compatibilitat entre navegadors de funcionalitats CSS, HTML5 i JS (http://caniuse.com/).

PhoneGap. Crea aplicacions per a mòbils amb tecnologia web (http://phonegap.com/).

The Languages And Frameworks That You Should Learn In 2015 (http://tutorialzine.com/2014/12/the-languages-and-frameworks-that-you-should-learn-in-2015/).

Web application frameworks a la Viquipèdia (https://en.wikipedia.org/wiki/Comparison_of_web_frameworks#PHP_2).

1. WordPress com a *framework* de desenvolupament web

Seguint el principi de *don't reinvent the wheel* (no reinventar la roda), el desenvolupament d'aplicacions es basa avui en dia en la reutilització de components ja existents. És el que podríem anomenar, emprant una terminologia més general, un *framework* de desenvolupament.

WordPress (WP) es pot veure com un *framework*, ja que ens proporciona de partida múltiples funcionalitats que podem aplicar als nostres projectes, encara que aquest projecte no sigui estrictament un blog o ni tan sols un web convencional.

Enumerem algunes d'aquestes funcionalitats que ja té incorporades WP, moltes de les quals són generals a totes les aplicacions:

1) Gestió d'usuaris: WP inclou un sistema complet d'autenticació d'usuaris amb registre, *login*, sistema per a recordar la contrasenya, etc. A més es poden definir rols específics. No és menys important que aquest sistema és segur i està en permanent monitorització per trobar forats de seguretat, ja que es fa servir en milions de webs d'internet.

2) Definició de l'estructura de continguts: WP inclou per defecte una estructura de continguts basada en pàgines i entrades, però es pot ampliar fàcilment a través de la seva API per definir nous tipus de continguts i camps a mida. A més inclou un potent sistema de taxonomies vinculades al contingut que també es pot estendre mitjançant la seva API.

3) Personalització de rutes: WP permet personalitzar les rutes a través de la seva API a través de funcions com `add_rewrite_rule`.

4) Gestió de dades: disposa d'un sistema de gestió de dades que permet, per exemple, desar paràmetres de configuració o qualsevol altre contingut a la base de dades.

5) Gestió de mèdia: sistema per a gestionar, a través de l'escriptori de WP, imatges i tota mena d'arxius. A més d'un sistema per a redimensionar les imatges a qualsevol mida (amb *crop* o sense *crop*).

Hem vist a la llista anterior que hi ha moltes funcionalitats que podem aprofitar, encara que en un projecte en concret és probable que calgui configurar-ne o modificar-ne correctament algunes, ja que en principi estan pensades per al propòsit pel qual va ser pensat WP inicialment, la gestió d'un blog.

Així, per exemple, els rols d'usuari i permisos associats estan pensats per a gestionar una publicació electrònica. Si l'aplicació que volem desenvolupar té una estructura diferent caldrà modificar aquests rols i permisos, però podrem conservar en canvi tot el funcionament del sistema de gestió d'usuaris, que és gran part de la complexitat del problema. En qualsevol cas serà molt més fàcil que crear un sistema de gestió d'usuaris rols i permisos des de zero.

L'accés a totes aquestes funcionalitats de WP es fa a través de les diferents API que formen part del seu nucli (*core*). Les API de WordPress ens permeten accedir a totes aquestes funcionalitats sense comprometre la seguretat de la plataforma i la seva evolució. Suposen una capa d'abstracció per sobre del nucli de WP.

WP no té estrictament una arquitectura MVC, però s'hi assembla. Per exemple, la part dels temes (*themes*) es correspondria a la part View de MVC, i el Model és en certa manera la base de dades.

En realitat WordPress, més que una arquitectura MVC, té una arquitectura basada en esdeveniments, podríem dir que és controlada per esdeveniments o *event-driven*. Un esdeveniment és qualsevol cosa que ha passat a l'entorn de WordPress, com per exemple que un usuari s'ha loginat, que un *plugin* s'acaba d'activar, que un usuari ha deixat un comentari, etc. En aquest moment, a través d'un *hook* a la nostra aplicació podem alterar el comportament per defecte de WP.

Hi ha dos tipus d'esdeveniments i *hooks* associats, les "accions" i els "filtres". El domini d'aquestes accions i filtres és el que ens permetrà treure el màxim rendiment del desenvolupament d'aplicacions amb WP.

1.1. Definició de tipus de continguts, camps personalitzats i taxonomies

El model de dades de WordPress, d'entrada, és molt senzill. Es basa en tres tipus de contingut, entrades (*posts*), pàgines (*pages*) i arxius (*attachments*), i dues taxonomies (categories i *tags*) vinculades a les entrades. Aquest model està pensat per un web convencional de tipus blog i en aquest cas és més que suficient.

Però lògicament a la nostra aplicació web necessitarem definir un model de dades personalitzat. Afortunadament WP ens permet definir nous tipus de contingut, camps i taxonomies a través de la seva API.

1.1.1. Definició de tipus de contingut

A continuació veiem la definició d'un tipus de contingut nou (*custom post type* en la terminologia WP) fent servir el *hook* "init" per cridar una funció que registra un tipus de contingut nou amb l'*slug* (codi que l'identifica internament a WP) "producte".

```
add_action( 'init', 'create_post_type' );
function create_post_type() {
  register_post_type( 'producte',
    array(
      'labels' => array(
        'name' => __( 'Productes' ),
        'singular_name' => __( 'Producte' )
      ),
      'public' => true,
      'has_archive' => true,
    )
  );
}
```

Aquest codi, i en general qualsevol codi que estigui dirigit a definir l'estructura de continguts de la nostra aplicació, ha d'estar dins del codi del *plugin* o de l'arxiu `functions.php` del nostre tema. Com ja sabem, la sintaxi `__( 'Productes' )` serveix per a fer aquesta etiqueta traduïble. Idealment els texts traduïts a WP haurien de tenir un segon paràmetre que fa referència al domini de la traducció.

A l'hora de registrar un nou tipus de contingut hi ha moltíssimes opcions que podem definir, totes documentades al *còdex* de WP.

Per ampliar informació

Sobre el sistema de traducció de WP i el domini de la traducció: https://codex.wordpress.org/Function_Reference/_2

Per registrar nous tipus de contingut:

https://codex.wordpress.org/Function_Reference/register_post_type

Un cop registrat el nou tipus de contingut veurem que apareix una nova opció a l'escriptori per poder crear, editar o esborrar els continguts d'aquest tipus, de la mateixa manera que ho podem fer amb les entrades o pàgines.

Però ens falta saber com podem mostrar aquests continguts al *frontend*. Per a aquest objectiu el millor és fer servir un *loop* personalitzat, filtrant per aquest tipus de contingut, com veiem al següent exemple:

```
$args = array( 'post_type' => 'producte', 'posts_per_page' => 10 );
//loop personalitzat que mostra 10 continguts de tipus producte
$loop = new WP_Query( $args );
//a partir d'aquí tenim el loop normal de WP
while ( $loop->have_posts() ) : $loop->the_post();
  the_title();
  echo '<div class="entry-content">';
  the_content();
```

```
    echo '</div>';
  endwhile;
```

Hi ha algunes plantilles que ja estan predefinides pels tipus de contingut personalitzats (*custom-types*) a la jerarquia de plantilles de WordPress. Són les següents:

1) La plantilla de la pàgina única del contingut: `single-{post-type}.php`

2) La plantilla de l'arxiu (llistat de tots els continguts) d'aquest tipus de contingut: `archive-{post-type}.php`

Aquest arxiu (l'URL on podem visualitzar el llistat) existirà només si el paràmetre `'has_archive'=>true`. En aquest cas l'URL de l'arxiu serà l'*slug* del tipus de contingut (a l'exemple anterior seria `/producte`). Si el paràmetre `'has_archive'` és una cadena de text, per exemple `'llistat_productes'`, l'URL de l'arxiu serà aquest. Cal fer atenció perquè aquestes URL no funcionaran fins que es refresqui l'opció *permalinks* de l'escriptori de WP: és en aquest moment quan WP refà les regles de reescriptura i per tant en general qualsevol canvi que impliqui canvis en els URL implicarà que refresquem les opcions d'aquesta pàgina.

Si volem escriure l'URL d'un arxiu per un tipus de contingut a una plantilla podem fer servir la següent expressió:

```
<?php echo get_archive_link('producte'); ?>
```

Per ampliar informació

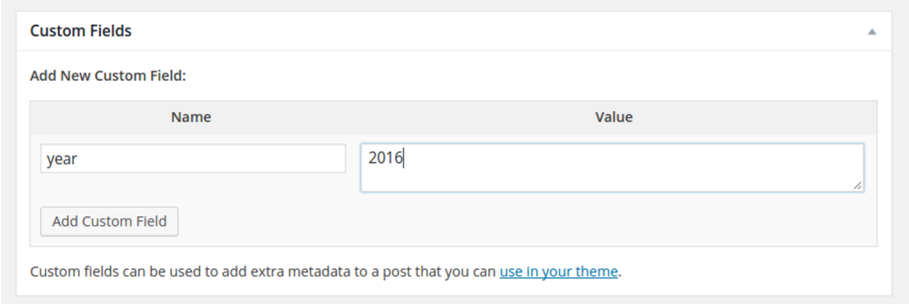
Tipus de contingut personalitzats: https://codex.wordpress.org/Post_Types

1.1.2. Definició de camps personalitzats

A l'apartat anterior hem vist com crear tipus de contingut personalitzats, però aquests continguts per defecte només tenen un camp títol i un camp descripció (*body*) i sovint necessitarem camps addicionals. WordPress ens permet amb la seva API definir tants camps personalitzats com vulguem. És el que s'anomena en terminologia WP *custom field* o *post metadata*.

Aquesta opció es pot desplegar fàcilment a l'escriptori de WordPress quan estem editant un contingut i permet afegir tants camps com sigui necessari.

Opció *custom fields* en editar una entrada a WordPress



Un cop definits aquests camps els podem mostrar a la nostra plantilla fent servir els *tags* `the_meta()`, per tots ells, o `get_post_meta()` per un camp específic.

Encara que fàcil de fer, aquesta opció de creació de *custom fields* té grans limitacions:

1) Els camps estan associats individualment a cada contingut, i no a un tipus de contingut en general.

2) No hi ha control sobre el tipus de contingut del camp, com per exemple dates, arxius o imatges. Només podem emmagatzemar texts i sense cap mena de validació.

3) La interfície d'usuari és molt bàsica.

Per ampliar informació

Sobre `get_post_meta()`, al *còdex*: https://developer.wordpress.org/reference/functions/get_post_meta/

Per aquests motius és altament recomanable fer servir alguna solució externa que arregli aquests defectes, un candidat principal a fer-ho és CMB2 (https://wordpress.org/plugins/cmb2/). CMB2 és un *plugin*/llibreria que millora de manera molt clara tots els problemes anteriors. Aquesta llibreria també es pot integrar dins d'un tema o un altre *plugin* de manera independent de manera que facilitem la instal·lació als usuaris.

Per ampliar informació

Sobre els camps: https://github.com/WebDevStudios/CMB2/wiki/Field-Types

Amb CMB2 podem definir camps específics per qualsevol tipus de contingut en forma de *metaboxes*. Els *metaboxes* formen part de l'API de WP i permeten definir visualment la interfície de cada camp. CMB2 permet gestionar-ho de manera molt fàcil. Hi ha més de trenta tipus de camps diferents definits.

Per exemple, tenim el tipus *file* per pujar imatges o arxius, *textarea* per editar textos amb un editor de tipus *wysiwyg*, *colorpicker* per triar colors, etc.

Veiem un exemple que ens permetria afegir el camp "preu" i "referència" al contingut personalitzat "producte".

```
$prefix = '_demo_';  //cal un prefix pels camps personalitzats
$cmb_demo = new_cmb2_box( array(
 'id'            => $prefix . 'metabox',
 'title'         => __( 'Test metabox', 'cmb2' ),
 'object_types'  => array( 'producte', ), // Post types on aplica
) );
$cmb_demo->add_field( array(
 'name'       => __( 'Preu', 'cmb2' ),
 'desc'       => __( 'Preu en euros', 'cmb2' ),
 'id'         => $prefix . 'preu',
 'type'       => 'text',
 // 'repeatable'      => true,  //pot ser un camp replicable
) );
```

```
$cmb_demo->add_field( array(
 'name'       => __( 'Referència', 'cmb2' ),
 'desc'       => __( 'Codi de la referència', 'cmb2' ),
 'id'         => $prefix . 'ref',
 'type'       => 'text',
 // 'repeatable'      => true,  //pot ser un camp replicable
) );
```

Per visualitzar aquests camps al nostre tema o *plugin*:

```
// Obtenir les dades a partir del prefix i l'ID del post
$text = get_post_meta( get_the_ID(), '_yourprefix_text', true );

// Mostrar el contingut del camp
echo esc_html( $text );
```

Per ampliar informació

Plugin CMB2: https://wordpress.org/plugins/cmb2/

Repositori CMB2 a GitHub: https://github.com/WebDevStudios/cmb2

1.1.3. Definició de taxonomies

Per acabar de definir una bona arquitectura de continguts ens cal poder definir noves taxonomies i poder-les assignar als tipus de contingut. Per a aquest propòsit també tenim una crida de l'API a WordPress, `register_taxonomy()`.

A continuació veiem un exemple on registrem una taxonomia "tipus" pel nostre tipus de contingut producte.

```
function custom_taxonomy() {
 $labels = array(
 'name'                       =>  'tipus', //slug taxonomia
 'singular_name'              => 'Tipus de producte',
 'menu_name'                  => 'Tipus de producte',
);
$args = array(
 'labels'                     => $labels,
 'hierarchical'               => false,
 'public'                     => true,
 'show_ui'                    => true,
 'show_admin_column'          => true,
 'show_in_nav_menus'          => true,
 'show_tagcloud'              => true,
);
register_taxonomy( 'tipus', array( 'producte' ), $args );
}
add_action( 'init', 'custom_taxonomy', 0 );
```

Per ampliar informació

Sobre el registre de taxonomies: https://codex.wordpress.org/Function_Reference/register_taxonomy

1.1.4. Mides d'imatge personalitzades

A WP hi ha tres mides d'imatge predeterminades: *thumbnail*, *medium* i *large*, la mida de cadascuna de les quals es pot canviar fàcilment a través de l'escriptori (paràmetres/*media*). WordPress crea imatges al directori *uploads* en aquestes tres mides automàticament a partir de cada original que hi pugem.

Amb la funció `add_image_size` podem definir mides noves per a les imatges.

Abans de fer-ho ens hem d'assegurar a l'arxiu `functions.php` del nostre tema que estigui activat el suport a *post-thumbnails*:

```
add_theme_support('post-thumbnails');
```

Llavors podem crear noves mides de la següent manera:

```
add_image_size('mini', 120, 120, true);
add_image_size('big', 1200, 900, false);
```

Al primer cas hem creat una nova mida amb el nom "mini" que fa 120px x 120px i amb el quart paràmetre *crop = true*. Amb *crop* activat WP retallarà la imatge a la mida indicada exacta encara que sigui perdent la relació d'aspecte. En canvi si el paràmetre és *false* la imatge es retallarà fent servir la mida més gran i conservant la relació d'aspecte.

Una cosa a tenir en compte és que un cop hem definit una mida especial d'imatge, si canviem aquestes mides posteriorment, o el paràmetre *crop*, les imatges creades fins al moment no es redimensionaran automàticament. Per tal de fer-ho caldrà instal·lar algun *plugin* especial, com Regenerate thumbnails (https://wordpress.org/plugins/regenerate-thumbnails/).

A WP cada contingut pot tenir una imatge destacada, per mostrar aquestes imatges a les nostres plantilles només caldrà escriure, en el cas per exemple de la mida definida anteriorment "mini":

```
<?php the_post_thumbnail('mini')?>
```

1.1.5. Exemple complet

Veiem a continuació un exemple complet on es defineix un tipus de contingut "producte", que es pot consultar a través de l'API REST i té imatge destacada. Els productes tenen dos camps personalitzats (preu i referència) i una taxonomia associada anomenada "tipus de producte".

```
/* incloure llibreria cmb2, necessària per definir els metaboxes (camps personalitzats)*/

require "cmb2.php";
/* cal comentar o eliminar a aquest arxiu els camps d'exemple que hi ha ara associats a
'page' i altres continguts */



/* crear tipus de contingut producte */

add_action( 'init', 'create_post_type' );

function create_post_type() {
```

```
  register_post_type( 'producte',
    array(
      'labels' => array(
        'name' => __( 'Productes' ),
        'singular_name' => __( 'Producte' )
      ),
      'public' => true,
      'has_archive' => 'llistat_productes',
      'supports' => array( 'thumbnail','title', 'editor', 'custom-fields' )
    )
  );
}



/*  crear taxonomia tipus de producte */

function custom_taxonomy() {

 $labels = array(
  'name'                       =>  'tipus', //slug taxonomia
  'singular_name'              => 'Tipus de producte',
  'menu_name'                  => 'Tipus de producte',

 );
 $args = array(
  'labels'                     => $labels,
  'hierarchical'               => false,
  'public'                     => true,
  'show_ui'                    => true,
  'show_admin_column'          => true,
  'show_in_nav_menus'          => true,
  'show_tagcloud'              => true,
 );
 register_taxonomy( 'tipus', array( 'post','producte' ), $args );

}
add_action( 'init', 'custom_taxonomy', 0 );

/* crear camps preu i referència  */

add_action( 'cmb2_admin_init', 'exemple_register_demo_metabox' );
/**
 * és recomanable canviar el prefixe
 */
function exemple_register_demo_metabox() {
```

```
// Start with an underscore to hide fields from custom fields list
$prefix = '_exemple_demo_';

/**
 * Metabox amb 2 camps
 */
$cmb_demo = new_cmb2_box( array(
 'id'            => $prefix . 'metabox',
 'title'         => __( 'Informació producte', 'cmb2' ),
 'object_types'  => array( 'producte', ), // Post type
) );

$cmb_demo->add_field( array(
 'name'       => __( 'Preu', 'cmb2' ),
 'desc'       => __( 'Preu en euros', 'cmb2' ),
 'id'         => $prefix . 'preu',
 'type'       => 'text_small',
) );

$cmb_demo->add_field( array(
 'name' => __( 'Referència', 'cmb2' ),
 'desc' => __( 'Codi de referència del producte', 'cmb2' ),
 'id'   => $prefix . 'ref',
 'type' => 'text',
 // 'repeatable' => true,
) );

}
```

Activitat

Feu el següent exercici a partir de l'exemple anterior: es vol crear un sistema de gestió de pel·lícules amb WordPress. Per aquest propòsit definirem un nou tipus de contingut *movie* amb els següents camps:

1) Títol (text curt)

2) Resum (text llarg)

3) Director (text curt)

4) Duració (numèric, minuts)

5) Any (numèric)

6) Gènere (taxonomia)

7) Imatge destacada

8) Tràiler (text curt, URL del vídeo)

Es demana definir el tipus de contingut i els camps o taxonomies associats.

Per ampliar informació

Trobareu el codi de l'exemple a GitHub (https://github.com/danieljulia/backend_wp_custom_types_taxonomy_metaboxes/).

1.2. Definició de nous rols i permisos

Una de les funcionalitats més freqüents a qualsevol aplicació és la gestió d'usuaris, amb tot el que això implica: autenticació, gestió de rols i permisos, gestió de contrassenyes, etc. WordPress, mitjançant el mòdul de gestió d'usuaris, ja ho té implementat, de manera que ho podem aprofitar a la nostra aplicació.

Hem de tenir en compte, però, que WordPress està pensat inicialment com una plataforma de *blogging* i que per tant els rols d'usuaris estan orientats en aquest sentit. Així, per exemple, el rol *contributor* pot escriure els seus propis *posts* però no els pot publicar. Probablement voldrem modificar aquest funcionament. Per tal de fer-ho podem afegir rols mitjançant `add_role` i capacitats (sempre associades a rols) amb `add_cap` o eliminant els ja existents amb `remove_role`.

Per ampliar informació

Llista completa de rols i capacitats per defecte a WP:
https://codex.wordpress.org/Roles_and_Capabilities

En concret, a l'API trobem les següents funcions:

1) `add_role()`: permet definir un nou rol

2) `remove_role()`: permet eliminar un rol

3) `add_cap()`: permet afegir una "capacitat" a un rol

4) `remove_cap()`: permet eliminar una "capacitat" a un rol

5) `get_role()`: retorna informació del rol i les "capacitats" associades.

Veiem, per tant, que no només podem crear rols personalitzats sinó també "capacitats" personalitzades. En aquest sentit tenim molt de control sobre la gestió dels usuaris. A continuació tenim un exemple en què definim un rol nou amb una capacitat nova.

```
$result = add_role( 'usuari_especial', 'Usuari especial', array(
    'read' => true, // True activa aquesta capacitat de WP
    'edit_posts' => true,
    'delete_posts' => false, // false la desactiva
    'qualsevol' => true //pot ser una capacitat nova
));
```

1.3. CMS *headless* i *decoupled*

Qualsevol CMS es pot dividir funcionalment de manera bastant clara en dos vessants, el *backend* i el *frontend*. En el cas de WordPress:

1) El *backend* correspon a l'escriptori, que serveix per a gestionar tot el contingut. Permet gestionar els continguts, crear, editar o eliminar dades.

2) El *frontend* correspon al tema. S'encarrega de mostrar els continguts a l'usuari, normalment a través d'un sistema de temes i plantilles, i ens permet navegar pels continguts.

Parlem d'un CMS *headless* quan només s'ha deixat el *backend* i s'ha substituït la part del *frontend* per un altre sistema diferent, que no forma part, per força, del propi CMS.

Veiem a continuació alguns tipus de projecte on és possible que ens interessi aprofitar la part del *backend* d'un CMS i al mateix temps implementar una estratègia diferent pel *frontend*, per mostrar les dades a l'usuari d'una altra manera, per exemple fent servir AngularJS o React per a la visualització dels continguts.

Alguns casos d'ús on es podria aplicar aquesta idea de CMS *headless*:

1) Aplicació de mòbil: en el cas que tinguem una aplicació per a mòbils el CMS *headless* pot servir per a gestionar els continguts de l'aplicació, mentre que la part del *frontend* pot ser una aplicació nativa o híbrida (en qualsevol plataforma com iOS, Android, etc.).

2) Optimització de la càrrega de la web: seguint el sistema tradicional en la majoria de CMS, cada contingut que és consultat per un usuari implica la càrrega d'una pàgina HTML sencera. Aquesta pàgina conté molts elements comuns (capçalera, peu, blocs, etc.), i molts recursos (llibreries JavaScript, CSS) que s'han de tornar a carregar a cada petició. Tots aquests arxius i recursos fan que el temps de càrrega no sigui òptim. Implementant la part del *frontend* amb un sistema AJAX, de manera que només es sol·liciti al servidor la informació que realment cal actualitzar, millorarà molt el temps de resposta.

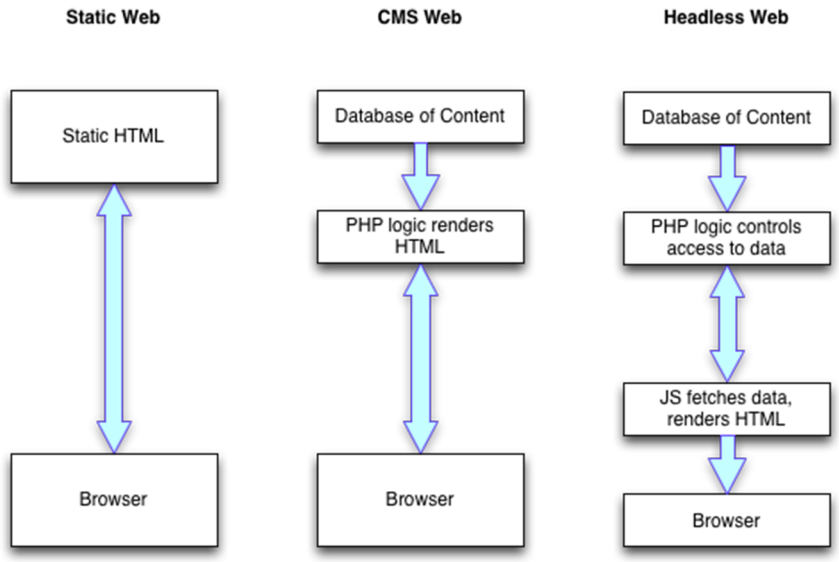
3) Si el que volem és proporcionar una **API de continguts** que funcioni com un servei web tampoc necessitem la part del *frontend*. En l'era actual de les dades i els objectes interconnectats (*Internet of Things*) ens trobem cada vegada més amb projectes que no corresponen amb la idea tradicional de consumir els continguts a través del web. En aquest cas el CMS ens pot servir per crear aquest servei web d'una manera relativament senzilla.

4) Hi ha **altres casos** on pot ser interessant fer servir un CMS *headless*. Posem com a exemple el cas del web Inside/Out del MOMA.

Web Inside/Out

Aquest web fa servir WP en mode *headless* perquè hi havia un requisit tècnic que obligava a fer servir el llenguatge Ruby a la part del *frontend*. Per aquest projecte es va desenvolupar un *plugin* a mida per servir el contingut de WP en format JSON.

Al següent gràfic podem veure tres esquemes de funcionament web client-servidor, des del més bàsic (web estàtic), seguit del tradicional (CMS) i finalment el *headless*. Cal tenir en compte que a la versió *headless* cal una lògica de navegació implementada en JavaScript.



Font: https://www.ostraining.com/blog/drupal/what-is-headless-drupal/

Pros i contres de l'estratègia *headless*

Pros:

1) És molt més eficient. Com que no cal fer tantes peticions HTTP, la velocitat de càrrega de les pàgines és molt més gran, i la carrega del servidor inferior.

2) Tenim flexibilitat total en la part del *frontend* i, de fet, podem tenir tant un web com una aplicació nativa a dispositius mòbils.

Contres:

1) Atès que les pàgines de contingut normalment no tenen una pàgina única el rastreig dels cercadors no serà tan eficient.

2) En fer servir un CMS pel *backend* caldrà més desenvolupament a mida a la part del *frontend* perquè no aprofitarem el sistema de plantilles que ens proporciona el gestor.

3) En ser una estrategia relativament nova, és molt probable que la tecnologia evolucioni i canviï els propers temps, amb la qual cosa la nostra implementació del *frontend* podria quedar poc actualitzada al futur.

Per ampliar informació

Sobre l'estratègia *headless*: *Plugin* que desactiva la part *frontend* de WP (*cup-headless*): https://github.com/ryan-mark/wp-headless

Resum del concepte i recursos associats: "Decoupled CMS: Why "Going Headless" Is Becoming So Popular": https://pantheon.io/decoupled-cms

"Create a Mobile Application Using WordPress, Ionic, and AngularJS": http://code.tutsplus.com/tutorials/create-a-mobile-application-using-wordpress-ionic-and-angularjs--cms-24170

1.4. Les API de WordPress

Sempre que fem servir WP com a *framework* de desenvolupament d'aplicacions necessitarem conèixer bé quines són les API que podem utilitzar al nostre projecte. Les API de WordPress són els punts d'entrada que ens permeten tenir accés a totes les funcionalitats de la plataforma.

Això també és important a l'hora de desenvolupar *plugins*. Les API afegeixen una capa d'abstracció de manera que ens asseguren que, fent-les servir, no causarem problemes de seguretat i que les funcionalitats s'aniran mantenint a mesura que apareixen noves versions. Ens aïllen de la implementació interna del codi.

Les API estan classificades en diferents seccions segons el tipus de funcionalitat. Així trobem, per exemple, l'API per als *shortcodes*, que ens permet programar aquesta funcionalitat de WP, o l'API de la base de dades que permet desar i recuperar continguts, l'API de *widgets* que ens permet crear *widgets* per a l'escriptori de WordPress, entre d'altres. A continuació comentarem algunes de les més importants classificades per tipus.

1.4.1. Mòdul de gestió d'usuaris

Aquest mòdul s'encarrega de gestionar els perfils d'usuari, l'autenticació del web, els diferents permisos i rols associats, etc.

1.4.2. Gestió d'arxius (*file management*)

El Media Uploader, integrat a l'escriptori de WP, és una eina molt útil per a gestionar arxius.

A més a més, podem fer servir la funció `media_handle_upload` per pujar imatges o qualsevol tipus d'arxiu al gestor de mèdia també des del *frontend*. Aquesta funció permet pujar diversos arxius simultàniament.

Per al cas concret de les imatges tenim la possibilitat de definir mides especials amb la funció `add_image_size`, com ja hem vist anteriorment.

1.4.3. Sistema de plantilles (*template management*)

WP disposa d'un potent sistema de plantilles que permet crear pàgines personalitzades fins al nivell que vulguem.

1.4.4. Sistema de base de dades (*database management*)

La interacció amb la base de dades es divideix en tres API:

1) Options API: serveix per a emmagatzemar dades genèriques a la base de dades.

2) Transients API: serveix per a guardar dades temporalment. És molt similar a l'anterior però amb un temps de caducitat al cap del qual les dades s'esborren.

3) Metadata API: serveix per a afegir metadades als continguts o usuaris.

A més existeix un objecte per a gestionar la memòria cau `WP_object_cache`, però cal remarcar que aquesta memòria només es fa servir durant la petició d'una pàgina.

WordPress no té un sistema de *caching* permanent, això només es pot aconseguir a partir de *plugins* externs.

Si volem implementar una cau (o *cache*) a la nostra *webapp* ho podem aconseguir amb l'API Transient que ens permet desar a la base de dades continguts durant un temps determinat.

1.4.5. Rutes (*Routing*)

Ens permet definir les nostres pròpies rutes a través del sistema de gestió de rutes del servidor web Apache.

Per ampliar informació

Llibreria que simplifica el procés: https://github.com/jbrinley/WP-Router

1.4.6. XML-RPC

Permet interactuar remotament amb WordPress des d'altres sistemes fent servir el protocol XML-RPC.

Per ampliar informació

https://codex.wordpress.org/Function_Reference/wp_schedule_event

1.4.7. Programació de tasques (*Scheduling*)

Una funcionalitat que possiblement tindrem a la nostra *webapp* és la programació de tasques específiques. Imaginem per exemple un procés que ha de cridar un servidor per recuperar informació cada cert temps. WP també ens ofereix un mecanisme per a programar tasques. Cal remarcar, però, que només

funcionarà si el web té visites actives: si no hi ha cap visita WP no té manera de fer les crides necessàries. En aquest cas caldrà cridar l'adreça `/wp-cron.php` al *crontab* del nostre sistema, crida que activarà totes les tasques pendents.

1.4.8. *Plugins*

L'API de *plugins* és la que ens permet ampliar les funcionalitats de WP. Un *plugin* normalment utilitza moltes altres API per aconseguir el seu objectiu.

Per ampliar informació

Instruccions detallades per escriure un plugin: https://codex.wordpress.org/Writing_a_Plugin

1.4.9. Configuració de l'escriptori

L'escriptori de WordPress és una eina de gestió que fa molts anys que funciona. Està testada en milers de webs i té un nivell òptim d'usabilitat. Podem treure profit d'aquesta eina i configurar-la per a les necessitats de la nostra aplicació.

Les anteriors són només algunes de les més importants, al *còdex* podeu trobar la llista sencera d'API de WordPress (https://codex.wordpress.org/WordPress_APIs).

1.5. *Hooks*, filtres i accions

WP com a *framework* és una plataforma basada en esdeveniments. Podem interceptar aquests esdeveniments amb un *hook* de dues maneres, a través d'accions o de filtres.

Les accions són un tipus de *hook* que signifiquen que alguna cosa ha passat, en aquest moment la nostra aplicació pot registrar el seu propi codi o modificacions. Com que hi ha moltes accions hem de ser conscients de les conseqüències en el rendiment que pot tenir fer-ne servir una o altra.

Per exemple, l'acció `wp_footer` s'executa en el moment que WP mostra el peu del web. Si l'interceptem podem afegir codi HTML o incloure un fragment de JavaScript. Hi ha centenars d'accions documentades.

Per ampliar informació

Sobre les accions: https://codex.wordpress.org/Plugin_API/Action_Reference

A continuació veiem un exemple on fem servir el *hook* "`publish_post`" per cridar un mètode estàtic d'una classe *emailer* que envia un correu electrònic cada cop que es publica un *post* (cal observar que a l'*array* el primer paràmetre és el nom de la classe i el segon el nom del mètode estàtic, però podria ser només un paràmetre amb el nom d'una funció):

```
class emailer {
  static function send($post_ID)  {
    $friends = 'bob@example.org,susie@example.org';
   mail($friends,"sally's blog updated",'I just put something on my blog: http://blog.example.com');
    return $post_ID;
  }
}
```

```
add_action('publish_post', array('emailer', 'send'));
```

El format general de la funció `add_action`:

```
add_action ( 'hook_name', 'your_function_name', [priority], [accepted_args] );
```

Els filtres, per una altra banda, són un tipus de *hook* que ens permet rebre un determinat fragment de dades, manipular-lo, i després tornar-lo de nou a WordPress abans que es mostri al navegador.

Un "filtre" permet interceptar el contingut abans que aquest contingut es mostri a l'usuari o es desi a la base de dades.

> Un exemple de filtre seria `the_content`, que s'activa just abans del moment en què es mostra el contingut a l'usuari. Amb aquest filtre podríem afegir qualsevol cosa al contingut, com per exemple un fragment HTML amb el nom de l'autor.

També hi ha centenars de filtres perfectament documentats al *còdex* (https://codex.wordpress.org/Plugin_API/Filter_Reference).

Veiem un exemple d'ús d'un filtre on es filtren determinades paraules quan un usuari deixa un comentari (al filtre `comment_text`):

```
function filter_profanity( $content ) {
 $profanities = array('badword','alsobad','...');
 $content = str_ireplace( $profanities, '{censored}', $content );
 return $content;
}
add_filter( 'comment_text', 'filter_profanity' );
```

La forma general és:

```
add_filter ( 'hook_name', 'your_filter', [priority], [accepted_args] );
```

1.6. *Widgets*

Un *widget* és un bloc de contingut que l'administrador del web pot situar a qualsevol *sidebar* del nostre tema, fent servir la opció Widgets de l'escriptori.

La creació de *widgets* es fa a través de l'API de Widgets. Consisteix a definir una classe derivada de la classe base `WP_Widget`.

El mètode `widget` és el que mostra el contingut al *frontend*. Si el *widget* té opcions cal definir el seu *backend* a través del mètode `form` (la part que es veu a l'escriptori i que permet que l'administrador variï certs paràmetres i opcions).

Veiem a continuació un exemple on podem definir un títol a mida per al *widget*, que es pot canviar. De la mateixa manera podríem afegir tants camps configurables com necessitem.

```
<?php

// Widget molt senzill

//definició
class simple_widget extends WP_Widget {
function __construct() {

 parent::__construct(
 // ID del widget
 'simple_widget',
 // El nom que apareix a l'escriptori, traduible amb el domini 'domini'
 __('Widget simple', 'domini'),
 // La descripció
 array( 'description' => __( 'Un widget molt simple', 'domini' ), )
 );
}
// Creació del front-end del widget
// Això és el que es veu al frontend

public function widget( $args, $instance ) {
 $title = apply_filters( 'widget_title', $instance['title'] );
 // before and after widget arguments are defined by themes
 ?>
 <!-- aquí comença el widget -->
 <?php echo $args['before_widget']; ?>
 <?php if ( ! empty( $title ) )
 echo $args['before_title'] . $title . $args['after_title'];
 ?>
 <?php
 echo __( 'Hello, World!', 'domini' );
 echo $args['after_widget'];
 ?>
 <!-- final del widget -->
 <?php
}

// El Backend del widget
public function form( $instance ) {
 if ( isset( $instance[ 'title' ] ) ) {
  $title = $instance[ 'title' ];
 }else {
  $title = __( 'New title', 'domini' );
```

```
 }
 // Formulari
?>
 <p>
 <label for="<?php echo $this->get_field_id( 'title' ); ?>"><?php _e( 'Title:' ); ?></label>
 <input class="widefat" id="<?php echo $this->get_field_id( 'title' ); ?>" name="<?php echo
  $this->get_field_name( 'title' ); ?>" type="text" value="<?php echo esc_attr( $title ); ?>" />
 </p>
 <?php
}
// Actualitzant valors del widget
public function update( $new_instance, $old_instance ) {
 $instance = array();
 $instance['title'] = ( ! empty( $new_instance['title'] ) ) ? strip_tags( $new_instance
  ['title'] ) : '';
 return $instance;
}
}
//final de la definició
// Registra i carrega el widget
function load_widget() {
 register_widget( 'simple_widget' );
}
add_action( 'widgets_init', 'load_widget' );
```

1.7. *Shortcodes*

El sistema de *shortcodes* a WP permet que escrivint un codi determinat a la caixa de text d'un contingut, aquest codi es tradueixi en un contingut més complex. És una mena de "macro".

WordPress ja incorpora una sèrie de *shortcodes*, com per exemple `[audio]`:

```
 [audio mp3="source.mp3" ogg="source.ogg" wav="source.wav"]
```

L'API de *shortcodes* ens permet definir-ne de personalitzats. Definir un *shortcode* amb l'API és ben senzill. El codi pot estar tant a l'arxiu `functions.php` del tema com en un *plugin*.

```
//shortcode [demo]

function demo_func( $atts ){
 return "Hello world!";
}
add_shortcode( 'demo', 'demo_func' );
```

L'anterior codi fa que en escriure `[demo]` en un contingut de WordPress aquest es visualitzi com a *Hello world*!.

Sovint voldrem que el nostre *shortcode* tingui paràmetres:

```
// [demo foo="un valor"]
function demo_func( $atts ) {
    $a = shortcode_atts( array(
        'your_name' => 'John', //valor per defecte, hi poden haver més paràmetres
    ), $atts );

    return "Hello {$a['your_name']}!";
}
add_shortcode( demo, 'demo_func' );
```

En el cas següent:

```
 [demo your_name='Jess']
```

Es visualitzarà al *frontend* com `Hello Jess!`. I sense paràmetres `[demo]` farà servir el valor per defecte, `Hello John!`.

1.8. Desenvolupament de *plugins* amb WordPress

La definició de noves funcionalitats a WP passa per la creació d'un *plugin*. Aquest *plugin* normalment farà servir diferents API de WP per implementar les seves funcionalitats sense alterar el nucli del codi.

La manera de definir aquestes noves funcionalitats és a través dels *hooks*. Els *hooks* permeten interceptar el codi de WP en certs punts de manera que podem modificar el comportament per defecte, sense haver de tocar mai el nucli (*core*) del sistema. Aquests *hooks*, com ja hem vist prèviament, poden ser de dos tipus, filtres i accions.

Per entendre millor els conceptes de filtres i accions implementarem un *plugin* senzill. Per començar ens fixarem en el *plugin* d'exemple, anomenat "Hello Dolly", que existeix a qualsevol instal·lació de WordPress. De fet, és un *plugin* extremadament simple i consta d'un únic arxiu. L'efecte d'activar aquest *plugin* és l'aparició de cites aleatòries a la part superior dreta de l'escriptori de WP.

Normalment els *plugins* són molt més complexos, contenen diversos arxius (PHP, CSS, JS, recursos) i per tant el que es fa és crear una nova carpeta dins de `/plugins` on s'organitzen tots. Aquesta carpeta ha de tenir un nom descriptiu i que no es pugui confondre amb altres *plugins* existents (recordem que hi

ha milers de *plugins* i per tant molts dels noms ja existeixen). Dins d'aquesta carpeta hi dipositarem tots els arxius. Un d'ells, el principal, contindrà les definicions en forma de comentari de PHP.

Com a exemple veiem la definició del *plugin* "Hello Dolly":

```
/**
 * @package Hello_Dolly
 * @version 1.6
 */
/*
Plugin Name: Hello Dolly
Plugin URI: http://wordpress.org/plugins/hello-dolly/
Description: This is not just a plugin, it symbolizes the hope and enthusiasm of an entire
generation summed up in two words sung most famously by Louis Armstrong: Hello, Dolly.
When activated you will randomly see a lyric from <cite>Hello, Dolly</cite> in the upper
right of your admin screen on every page.
Author: Matt Mullenweg
Version: 1.6
Author URI: http://ma.tt/
*/
```

Veiem que hi ha una sèrie de camps definits, aquests camps són especialment importants si volem publicar el nostre *plugin* al repositori oficial.

Activitat

Crearem un *plugin* que implementi alguns dels conceptes que hem explicat anteriorment. Anomenarem el nostre *plugin* Quick Instagram i tindrà les següents funcionalitats:

1) Generarà un *widget* que permetrà mostrar un nombre variable de fotos de qualsevol compte públic d'Instagram.

2) Un *shortcode* permetrà visualitzar aquestes fotos dins de qualsevol contingut escrivint, per exemple:

```
[instagram user='barackobama' num=5]
```

Activitat

Codi del *plugin* finalitzat a Github:
https://github.com/danieljulia/backend_wp_plugin_instagram

Es demana modificar l'exercici anterior per aconseguir les següents millores:

1) Guardar les consultes a Instagram a la base de dades de WP amb la Transients API (en cas contrari cada consulta al web implica una crida a l'API d'Instagram, que té limitacions).

2) Afegir una opció a l'escriptori de WordPress per tal de poder afegir la clau de l'API d'Instagram sense haver d'editar l'arxiu de configuració (es vol evitar que un usuari final hagi d'editar cap arxiu).

3) Es vol afegir un altre *widget* que mostri les fotos a partir d'un *hashtag*.

1.9. API REST WordPress

Un cas especial és l'API REST de WordPress, que està especialment pensada per a poder desenvolupar aplicacions *headless* i treure el màxim profit de la plataforma WP. Aquesta API és actualment un *plugin* però està previst que properament s'inclogui al nucli de WordPress.

WordPress és un bon CMS que ens permet fàcilment gestionar els continguts d'una aplicació, però el sistema està pensat per a consumir aquests continguts a través del mètode tradicional, les pàgines web en HTML/CSS a través del sistema de temes.

En alguns casos voldrem consumir aquests continguts d'altres maneres.

> Pensem per exemple en una aplicació mòbil en la qual els continguts es gestionen a través d'un WordPress en un servidor d'internet però es consumeixen a l'aplicació. En aquest cas disposar d'un servei per a sol·licitar la informació és just el que necessitem, i això és el que resol l'API REST de WordPress.

A través d'aquesta API podrem fer pràcticament qualsevol cosa que es pugui fer a través de l'escriptori de WordPress. No només consultar *posts* o pàgines, sinó publicar o editar nous continguts, afegir comentaris, etc.

Un cop instal·lat i activat el *plugin* podem fer crides REST de manera senzilla.

> Per exemple, la crida que cal per obtenir el contingut del *post* amb `id=1` seria `/wp-json/wp/v2/posts/1` (l'URL exacte pot canviar amb la versió del *plugin*)
>
> http://demo.wp-api.org/wp-json/wp/wp-json/posts/1

Si obrim aquest URL en un navegador amb una extensió per veure JSON, com per exemple JSONView per a Chrome, podrem veure l'arxiu JSON resultant:

Per ampliar informació

Extensió Jsonview per consultar arxius json al navegador Chrome: https://chrome.google.com/webstore/detail/jsonview/chklaanhfefbnpoihckbnefhakgolnmc.

Exemple de resposta JSON de la crida a l'API REST de WP


```
{
    id: 1,
    date: "2015-06-03T22:12:40",
    date_gmt: "2015-06-03T22:12:40",
  - guid: {
        rendered: "http://wp-api-de-elbinsta-n5pu020s13lb-1748419096.eu-west-1.elb.amazonaws.com/?p=1"
    },
    modified: "2015-06-03T22:12:40",
    modified_gmt: "2015-06-03T22:12:40",
    slug: "hello-world",
    type: "post",
    link: "http://demo.wp-api.org/2015/06/03/hello-world/",
  - title: {
        rendered: "Hello world!"
    },
  - content: {
        rendered: "<p>Welcome to WordPress. This is your first post. Edit or delete it, then start blogging!</p> "
    },
  - excerpt: {
        rendered: "<p>Welcome to WordPress. This is your first post. Edit or delete it, then start blogging!</p> "
    },
    author: 1,
    featured_image: 0,
    comment_status: "open",
    ping_status: "open",
    sticky: false,
    format: "standard",
  - _links: {
      - self: [
          - {
                href: "http://demo.wp-api.org/wp-json/wp/v2/posts/1"
            }
        ],
      - collection: [
          - {
                href: "http://demo.wp-api.org/wp-json/wp/v2/posts"
            }
        ]
```

Veiem alguns punts d'entrada de l'API (tots els URL van precedits de `wp-json/wp/v2/`):

1) Retornar tots els *posts*. Enviant una petició GET a /posts.

2) Retornar tots els *posts* d'un tipus de contingut determinat (per exemple, "producte"), /producte. En aquest cas cal indicar `show_in_rest=true` a la definició del tipus de contingut.

3) Retornar un post determinat /posts/28 (on 28 és l'`id` del *post*).

4) Retornar un llistat amb els tipus de contingut definits. GET /types.

5) Actualitzar usuari amb ID 4. Enviar una petició POST a /users/4.

6) Retornar tots els *posts* amb el terme "barcelona". GET /posts?filter[s]=barcelona.

Per motius de seguretat els camps personalitzats no estan disponibles per defecte a les crides de l'API. Per solucionar aquest problema podem fer servir el filtre `json_prepare_term` per afegir els camps que necessitem.

Per ampliar informació

La resta de les crides es troba a http://v2.wp-api.org/

Per ampliar informació

Sobre personalització dels camps de la resposta:

Afegir camps: https://1fix.io/blog/2015/06/26/adding-fields-wp-rest-api/

FIltrar resultats per taxonomies personalitzades: https://1fix.io/blog/2015/07/20/query-vars-wp-api/

2. Exemple d'aplicació

A continuació veiem un cas molt simple d'utilització de l'API REST de WordPress que ens permet consultar les entrades publicades.

El *frontend* consta només d'un arxiu `index.html` i un arxiu de JavaScript `app.js` que consulta les dades de Wordpress.

L'arxiu `app.js` és el següent:

```
//cal modificar aquest url i apuntar a la instal·lació de wp
var BASE_URL='http://example.com/wordpress';

var app = {

 init: function() {
  app.getPosts();
 },

 getPosts: function() {

  var rootURL = BASE_URL+'/wp-json/wp/v2';

  $.ajax({
   type: 'GET',
   url: rootURL + '/posts',
   dataType: 'json',
   success: function(data){

    $.each(data, function(index, value) {
     console.log(value);

         $('ul.topcoat-list').append('<li class="topcoat-list__item">' +

          '<h3>'+value.title.rendered+'</h3>' +
          '<p>'+value.excerpt.rendered+'</p></li>');
       });
   },
   error: function(error){
    console.log(error);
   }

  });
```

```
    }


}
```

Veiem que el programa fa peticions AJAX a l'API REST de WordPress i rastreja el resultat per escriure el contingut mitjançant crides amb jQuery.

Podeu veure l'exemple complet a GitHub (https://github.com/danieljulia/backend_wp_api_rest_v2_jquery_mobile).

Exercici

Amplieu l'aplicació de partida afegint les següents funcionalitats:

1) Mostrar al llistat les imatges destacades de cada entrada.

2) Afegir paginació (per exemple tres entrades per pàgina).

3) Afegir l'opció de veure el detall d'una entrada.

4) Afegir l'autor i la data de publicació de cada entrada.

Per ampliar informació

Documentació del *plugin* REST: https://wordpress.org/plugins/rest-api/

Documentació de l'API: http://v2.wp-api.org/

Exemple més complet d'integració: https://github.com/scottopolis/wp-rest-api-demo (a tall d'exercici, podeu adaptar aqueste exemple fet amb la v.1 a la v.2 de l'API REST de WordPress)

Un *framework* MVC per A WordPress: https://github.com/tombenner/wp-mvc

Useful Tips To Get Started With WordPress Hooks:

https://www.smashingmagazine.com/2016/01/get-started-with-hooks-wordpress/

3. Desenvolupament d'aplicacions web en Drupal

Drupal, igual que hem vist amb WordPress, és un potent *framework* de desenvolupament d'aplicacions.

En aquest apartat ens centrarem en la darrera versió de la plataforma, la 8, on s'han fet canvis encaminats especialment a aquest propòsit.

3.1. Drupal com a *framework*

Drupal és un dels CMS més potents de codi obert actuals. La versió 8, estrenada oficialment el novembre de 2015, és una reconstrucció total de les versions anteriors i incorpora components que el fan molt interessant com a *framework* de desenvolupament d'aplicacions.

Drupal 8 incorpora, entre d'altres coses, components de Symfony, un sistema integrat per a crear serveis web a través de REST, o un nou sistema de plantilles anomenat Twig (que també pertany a Symfony).

3.2. Millores de Drupal 8 respecte a la versió 7

Si ja coneixem Drupal 7 podrem comprovar com Drupal 8 ha evolucionat de manera molt significativa respecte a la versió anterior.

Per començar, molts dels mòduls contribuïts importants ara formen part del *core*, entre ells *views* o els d'internacionalització. A més a més, altres funcionalitats s'han incorporat al *core*, com un mòdul per a poder fer servir Drupal com un servei web REST.

Repassem algunes de les millores de la nova versió:

1) Tots els temes de Drupal són ara *responsive* i *mobile first*. També ho és el tema d'administració.

2) Incorpora un gestor d'idiomes i traduccions complet (aquesta era una de les funcionalitats que abans només era possible amb mòduls externs).

3) Sistema de control de configuració basat en arxius. Això simplifica molt el fet de moure els canvis del servidor de desenvolupament al de producció. Abans les dades de configuració estaven a la força dins de la base de dades.

4) Accessibilitat i web semàntic. Hi ha hagut diverses millores en aquestes àrees.

5) Integració del mòdul Views (vistes). El que probablement era el mòdul contribuït més important de Drupal 7 ara forma part del *core*. De fet, ara tant la portada del web com les pàgines d'administració són "vistes".

6) Serveis web incorporats. Aquests serveis permeten interactuar amb Drupal mitjançant JSON o XML.

7) Editor WYSIWYG incorporat. Encara que sembli mentida Drupal 7 no en tenia cap, seguint la filosofia Drupal es deixava aquest component a l'elecció del desenvolupador, però per defecte no n'hi havia.

8) Sistema de plantilles TWIG. Aquest sistema permet dissenyar els temes de manera molt més fàcil i segura. No cal fer servir etiquetes de PHP com abans.

9) Integració de diversos *frameworks* i llibreries. Entre ells diversos components del *framework* Symfony.

10) Millora del marcatge HTML5. Com per exemple els camps de tipus *e-mail*, *date* o *phone*.

Per ampliar informació

Trobareu una llista més exhaustiva d'aquestes funcionalitats a la documentació oficial de Drupal: https://www.drupal.org.

3.3. Drupal 8 enfocat a desenvolupament *backend*

Tot i aquests canvis Drupal 8 continua essent una eina estàndard de desenvolupament web en la forma clàssica, és a dir, un sistema que permet publicar pàgines en format HTML/CSS a partir d'un sistema de temes (plantilles).

Aleshores, com podem fer que Drupal es converteixi en una eina de desenvolupament d'aplicacions web?

Hem de pensar en Drupal com un sistema complex, amb multitud de components, dels quals podem aprofitar els que més necessitem. D'aquesta manera, segons la tipologia del projecte podem abordar el problema de maneres diferents. Una d'elles és la filosofia *headless* que ja hem vist al cas de WordPress.

3.4. Drupal 8 i Symfony

Seguint la filosofia de reutilitzar codi, Drupal 8 integra força components de Symfony. Entre aquests components hi trobem els següents:

1) HttpFoundation: el component que gestiona les peticions HTTP (*get*, *post*, etc...), capçaleres, redireccions, *cache* HTTP, etc.

2) HttpKernel: proporciona un procés estructurat per a convertir una sol·licitud en una resposta en fer ús del component EventDispatcher.

3) DependencyInjection: gestiona la creació d'objectes dins de l'aplicació.

4) EventDispatcher: proporciona eines que permeten l'enviament d'esdeveniments als components de l'aplicació que es comuniquen entre si.

5) Routing: mapeja una petició HTTP a una sèrie de variables de configuració.

6) Yaml: aquest component s'encarrega de carregar i desar arxius .yml.

3.5. Sistema de temes Twig

Un dels components de Symphony incorporats a Drupal 8 és el sistema de plantilles Twig. Les plantilles *twig* (amb extensió .html.twig) faciliten el desenvolupament de temes amb diverses millores respecte al sistema anterior basat en PHP (phptemplate).

Tot i que es tracta d'un nou sistema l'estructura és similar a la que hi havia a la versió 7. Així hi ha una millor adaptació dels desenvolupadors experimentats a la versió anterior de la plataforma.

En general les etiquetes que s'escrivien en PHP, ara s'escriuen amb el sistema Twig, intentant mantenir la nomenclatura. Per exemple, el següent codi escriu un enllaç a un *node* de Drupal 7:

```
<a href="/<?php print $node_url; ?>"><?php print $title; ?></a>
```

Mentre que ara amb el sistema Twig de Drupal 8 tenim:

```
<a href="/{{ node_url }}">{{ label }}</a>
```

Com podem veure ara resulta un codi molt més llegible i manipulable per perfils *frontend* que no cal que tinguin coneixements de PHP. De fet, les condicions, bucles i totes les estructures de programació també s'escriuen amb etiquetes especials de Twig. En general mai trobarem, ni és desitjable, cap rastre de codi PHP a les plantilles. Totes les marques `<?php ?>` són substituides per `{{ }}`.

Un dels grans avantatges de Twig és la possibilitat d'implementar herència a les plantilles, de manera que es redueix substancialment la redundància. Com sabem, la redundància pot ser un dels pitjors problemes de l'optimització de codi, ja que promou els errors.

La reutilització de plantilles s'aconsegueix a través de la instrucció `extends`, que permet que una plantilla sigui filla d'una altra:

```
{% extends "node.html.twig" %}
```

3.6. Classy

Classy és el tema que s'utilitza com a tema base per Bartik i Seven, els temes que hi ha per defecte en una instal·lació de Drupal. El seu propòsit és proporcionar classes per descriure els elements de la pàgina. Aquest tema base fa de pont entre les dades del *backend* i els temes finals. Per crear el nostre propi tema també podem aprofitar-lo, seguint la documentació al respecte.

Per ampliar informació

Documentació sobre el sistema de plantilles Twig: http://twig.sensiolabs.org/documentation

"Working With Twig Templates": https://www.drupal.org/node/2186401

En el cas que volguem un tema amb una quantitat mínima de classes naturalment també podem crear un tema des de zero, prescindint de Classy, o fer servir algun altre tema base o *framework*.

Per ampliar informació

Com crear un tema a partir de Classy: https://www.drupal.org/node/2165673

Com crear un tema

Per tal de desenvolupar un tema nou cal crear una carpeta dins de la carpeta *themes* i afegir la definició del tema a un arxiu especial. Aquest arxiu tenia abans (Drupal 7) l'extensió .info però ara és un arxiu .yml, tot i que l'estructura recorda també molt la versió anterior.

El nom de l'arxiu ha de ser igual que la carpeta, per exemple my_theme.info.yml, en el cas que estiguem utilitzant Classy com a tema base l'arxiu tindria la següent definició:

```
name: Theme name
description: Theme description
type: theme
core: 8.x
base theme: classy
```

En aquest arxiu de configuració del tema també hem de definir les regions (zones actives on podem assignar *blocks*, equivalents als *widgets* de WordPress), etc.

Una nova funcionalitat molt útil i interessant és la possibilitat d'activar als temes el *debugging* de Twig. Per tal d'aconseguir-ho simplement cal escriure (`debug: true`) a l'arxiu `sites/defaults/services.yml`. D'aquesta manera Drupal escriurà `<!-- comentaris html -->` indicant quina és la plantilla que correspon a cada fragment de pàgina generada al *frontend*, cosa que pot ser molt útil.

Per ampliar informació

Sobre Classy i *theming* a Drupal 8:

Theming Drupal 8: https://www.drupal.org/theme-guide/8

Working With Twig Templates: https://www.drupal.org/node/2186401

Using Classy as a base theme: https://www.drupal.org/theme-guide/8/classy

Documentació oficial sobre la definició de l'arxiu .info.yml: https://www.drupal.org/node/2349827

3.7. Desenvolupament de mòduls a Drupal

Drupal disposa d'un panell de control que permet configurar amb gran detall l'estructura de dades de la nostra aplicació, i definir les funcionalitats, els rols d'usuari, els permisos, la presentació, etc, sense haver de programar res ni instal·lar cap mòdul extern.

En casos particulars, però, ens podem trobar amb la necessitat d'haver de desenvolupar un mòdul específic per a implementar alguna funcionalitat concreta.

Amb la inclusió de components de Symfony al nucli de Drupal 8 la manera de desenvolupar mòduls ha canviat totalment respecte de la versió 7. El desenvolupament ara és totalment orientat a objectes i seguint el patró MVC (model vista controlador). Els mòduls, igual que a WordPress, tenen accés a l'API de Drupal per disposar de tota la funcionalitat del nucli.

La complexitat és molt més alta que a WordPress, a causa de dos factors. En primer lloc la integració de Symfony implica conèixer aquest entorn abans de començar a desenvolupar, i en segon lloc les funcionalitats de l'API són molt més nombroses que a WordPress, cosa que n'augmenta la dificultat.

Una bona estratègia per a crear un mòdul és partir d'algun altre ja existent, que ens servirà d'esquelet inicial, d'una manera similar al que faríem en crear un tema derivat, redefinint plantilles i definicions al full d'estils.

Per ampliar informació

Directori dels mòduls disponibles per Drupal: https://www.drupal.org/project/project_module/index

Exemple de mòdul bàsic de Drupal 8 amb codi a Github: http://getlevelten.com/blog/ian-whitcomb/drupal-8-module-development-part-1-getting-started

API de Drupal 8: https://api.drupal.org/api/drupal/8

Creating Drupal 8.x modules. Tutorial per crear mòduls: https://www.drupal.org/developing/modules/8

3.8. Drupal *headless*

Drupal incorpora tota una sèrie de mòduls (dins de la categoria "web services") que permeten implementar aplicacions *headless*, a través d'una API d'accés REST als continguts del CMS amb la qual podríem, per exemple, integrar Drupal amb un *frontend* implementat en sistemes com React o AngularJS.

Es pot utilitzar bàsicament per a dos tipus d'aplicacions:

1) Aplicacions mòbils: aplicacions que a través de jQuery Mobile i de la plataforma Cordova es poden convertir en aplicacions de les plataformes mòbils (iOS, Android, etc.).

2) ***Webapps headless***: aplicacions web en què tot funciona a través de JavaScript+HTML+CSS, i la navegació es fa exclusivament a través d'AJAX. En una *webapp headless* quan es navega pel contingut no hi ha peticions a altres URL a través del protocol HTTP, sinó que és el JavaScript que s'encarrega de fer una petició al servei web de Drupal i actualitzar el *frontend*. Aquest tipus d'aplicacions té l'avantatge de ser molt més ràpid.

DrupalGap (http://drupalgap.org/) és una iniciativa i un kit de desenvolupament de *webapps* per a Drupal que es basa en els serveis web de la plataforma. Intenta facilitar tots els passos per poder disposar més ràpidament d'una aplicació basada en Drupal al *backend*.

Per ampliar informació

Exemple de Drupal *headless*: https://github.com/joshkoenig/headless-drupal-angular

Informació sobre DrupalGap: http://drupalgap.org/

"Selecting a client side framework for Drupal":http://buytaert.net/selecting-a-client-side-framework-for-drupal

3.9. API Rest Drupal

El *core* (nucli) de Drupal 8 incorpora 4 mòduls relacionats amb la interfície REST.



1) HAL (Hypertext Application Language): és una convenció per a definir *hypermedia* a arxius en format JSON o XML.

2) HTTP Basic Authentication: permet autenticació i per tant no només consultar informació, sinó també afegir o editar continguts a Drupal a través de la interfície REST.

3) RESTful Web Services: el mòdul que permet l'activació dels serveis REST.

4) Serialization: permet l'exportació dels continguts en format JSON o XML.

La manera més flexible d'activar el servei REST a Drupal és a través d'una vista (*view*). D'aquesta manera podrem crear diferents punts d'entrada i configurar el format dels camps que volem exportar.

Per fer-ho seguirem els següents passos:

1) Activar almenys els mòduls RESTful Web Services i Serialization.

2) Configurar els permisos (*people*/*permissions*) dins de la secció REstful web services "Access GET on Content resource" per l'usuari anònim.

3) I finalment crear una *view* amb *display* "REST Export". En aquesta vista podrem triar els camps a mostrar, els filtres que volem aplicar (contextuals o no) i l'URL del punt d'entrada.

Si volem fer peticions POST haurem d'activar el mòdul HTTP Basic Authentication, i activar els permissos corresponents.

Autenticació

Les dues possibilitats d'autenticació per a l'API REST que proporciona el *core* són:

1) Cookie

2) HTTP basic

A mòduls "contribuïts" (no pertanyents al *core*) podem trobar altres tipus d'autenticació, com per exemple el molt utilitzat OAuth.

Per ampliar informació

Mòdul autenticació OAuth: https://www.drupal.org/project/oauth

Articles sobre Drupal 8:

Dries Buytaert. "Should we decouple Drupal with a client-side framework?": http://buytaert.net/should-we-decouple-drupal-with-a-client-side-framework

Joe Fender. "Ampliació: Introducció als web services de Drupal": https://drupalize.me/blog/201401/introduction-restful-web-services-drupal-8.

Travis Tidwell. "AngularJS with Drupal 8": https://events.drupal.org/losangeles2015/sessions/angularjs-drupal-8

Exemple d'integració de Drupal 8 amb AngularJS: https://github.com/clemens-tolboom/drupal-8-rest-angularjs

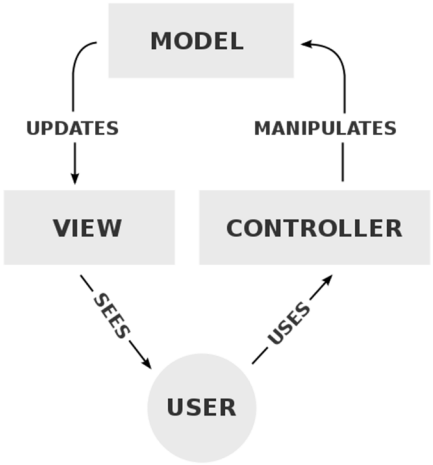
4. *Frameworks* MVC

La principal diferència entre un *framework* i un CMS és que el primer té molta més flexibilitat per a adaptar-se a qualsevol tipus d'aplicació. En canvi un CMS, sovint, precisament per facilitar el desenvolupament, està orientat a un tipus de projecte específic de manera que ja parteix amb una sèrie de funcionalitats per defecte.

La majoria dels *frameworks* de programació existents tenen una arquitectura MVC. L'arquitectura de programari MVC (sigles que volen dir Model/Vista/Controlador) separa el model de dades de la interfície d'usuari. És una manera d'organitzar el codi de manera que sigui més ordenat, eficient i reutilitzable.

En una plataforma MVC el model gestiona les dades de l'aplicació i les accions que es poden fer sobre aquestes dades, normalment mitjançant una base de dades. La vista proporciona la interfície d'usuari, que vindria a ser l'equivalent del tema als CMS. Finalment, el controlador rep les accions de l'usuari i fa crides als objectes del model i la vista per executar les accions apropiades.

A continuació veiem un esquema de funcionament d'un *framework* MVC:



Font: Viquipèdia

Els *frameworks* MVC ofereixen una flexibilitat molt més gran que el CMS, ja que estan pensats per a definir des de zero els seus blocs principals, començant pel model de dades, seguint amb els controladors i finalment amb la vista.

En el cas contrari, en un CMS com WordPress, atès que és una plataforma inicialment pensada per a construir blogs, encara que el podem adaptar a qualsevol tipus d'aplicació, com més diferent sigui del propòsit pel qual està pensant menys components estarem reutilitzant i més costós serà el desenvolupament.

4.1. Funcionalitats d'un MVC

Hi ha una sèrie de funcionalitats bàsiques que sovint requerim a les nostres aplicacions i que per tant haurien d'existir a qualsevol *framewok* MVC. Veiem a continuació algunes de les més importants.

1) Mapatge d'URL nets (URL *mapping*): en una aplicació web un dels requisits més importants és l'estructura d'URL i el mapa de navegació, per tant necessitem algun mecanisme que ens permeti dissenyar aquest mapa d'una manera fàcil i independent. També és important tenir mecanismes per a mostrar URL "nets". Per exemple, mostrar un URL del tipus "llista_productes", en lloc de "index.php?id=234124&op=234". Això és important per a l'accessibilitat i el posicionament en cercadors.

2) Accés a la base de dades: un bon *framework* també ha de facilitar eines que donin accés a la base de dades i fer-ho independentment del servidor concret de bases de dades que es faci servir (ja sigui MySQL, PostgreSQL, Oracle, etc.).

3) *Templates*: a la banda de la vista (*view*) és recomanable disposar d'un sistema de plantilles (o temes) per a facilitar la separació de contingut i presentació i per poder organitzar de la millor manera les plantilles.

4) Encauament (*caching*) de les pàgines per a augmentar-ne el rendiment: amb l'objectiu d'augmentar el rendiment del servidor l'encauament permet desar una versió ja construïda de les pàgines perquè en recuperar-la l'accés és molt més ràpid. El *framework* ha de permetre decidir com i quan fem servir aquest sistema d'encauament.

5) Autenticació d'usuaris: un *framework* ha d'incorporar la gestió d'usuaris amb totes les seves funcionalitats associades com són l'inici de sessió (*login*), registre, dades de perfil, rols i permisos d'usuaris, etc.

6) Seguretat: també és important que disposi de mecanismes de protecció contra atacs de *hackers*, *spam*, etc. Per exemple, haurem de tenir proteccions per a la injecció SQL o altres.

A més de les funcionalitats anteriors, per a desenvolupar aplicacions de *backend* potents és desitjable tenir eines auxiliars com les següents:

1) CRUD: sovint com a requeriment de la nostra aplicació tindrem una eina de gestió que ens permeti crear (*create*), llegir (*read*), actualitzar (*update*), i eliminar (*delete*) dades. D'aquestes inicials prové el terme CRUD. Les eines CRUD

permeten crear automàticament una eina de gestió dels continguts fent aquestes operacions bàsiques sobre les taules de la nostra base de dades. A partir d'aquestes operacions bàsiques podem redefinir o modificar-ne el funcionament, però bona part del desenvolupament ja estarà fet, estalviant-nos molt de temps. Sovint els *frameworks* disposen d'opcions o comandes CRUD que permeten generar un *backoffice* de gestió de continguts.

2) REST: d'altra banda és desitjable que el nostre *framework* disposi d'eines que permetin desenvolupar un servei web en mode REST. Un servei web REST permet operar sobre les dades a partir de peticions HTTP.

Imaginem que estem desenvolupant una aplicació destinada a mòbils. En aquest cas ens seran molt útils eines que permetin crear la interfície REST, amb la qual podrem actualitzar el contingut de l'aplicació, i per una altra banda de CRUD, que ens permetrà fer una eina de gestió per a les dades de l'aplicació.

A continuació veurem una breu introducció sobre tres *frameworks* de desenvolupament MVC basats en PHP.

De menys a més complexitat són CodeIgniter, Laravel i Symfony.

4.2. CodeIgniter

Dins de la gran família de *frameworks* de desenvolupament MVC basats en PHP CodeIgniter (a partir d'ara CI) és un dels més simples. La seva simplicitat té com a conseqüència que és fàcil d'aprendre però al mateix temps té com a contrapartida una limitació pel que fa a les funcionalitats.

Un projecte CI té una estructura d'arxius ben definida. Dins de la carpeta `application` trobarem les següents carpetes més importants on s'estructuren els arxius de la nostra aplicació:

1) Config: on es troben els arxius de configuració.

2) Controllers: on trobarem un arxiu per a cadascun dels controladors. Cada controlador actua com a ruta d'entrada i permet definir URL nets de l'estil `cataleg/producte/123`. Aquest seria el cas de tenir un controlador anomenat "catàleg" amb un mètode "producte" que té com a paràmetre un ID determinat.

3) Models: en aquesta carpeta trobarem un arxiu per a cadascun dels models de dades del projecte.

4) Views: en aquesta carpeta trobarem totes les plantilles i arxius associats (per exemple JS i CSS). A efectes pràctics correspon al que seria el tema d'un CMS com WP o Drupal.

5) REST: des de la versió 3 existeix la possibilitat de definir "verbs" a la definició de rutes, cosa que possibilita la creació d'interfícies REST.

Per ampliar informació

Sobre definició de verbs a les definicions de rutes: http://www.codeigniter.com/user_guide/general/routing.html#using-http-verbs-in-routes

CI té una configuració molt senzilla. De fet per fer-lo funcionar només cal copiar els arxius de partida. Opcionalment podem vincular una base de dades (especificant les dades de connexió), però per defecte no és obligatori ja que hi ha projectes en els quals és probable que no sigui necessari.

Un dels principals avantatges de CI és que no està subjecte a estrictes normes de programació i per tant es pot començar a fer servir ràpidament. Tampoc no hi ha obligació d'utilitzar cap llenguatge de plantilles, de manera que és molt ràpid entendre el seu funcionament.

Tots aquests factors donen lloc a una corba d'aprenentatge molt ràpida per a aquest *framework*.

Per ampliar informació

Web oficial del projecte CodeIgniter: http://www.codeigniter.com/.

4.2.1. Exemple CodeIgniter

Per entendre com funciona CI farem un exemple senzill, un web que mostri fotos d'Instagram.

La nostra aplicació tindrà els següents punts d'entrada:

1) Portada: a l'URL /. La portada de l'aplicació on podrem triar qualsevol de les dues opcions següents:

2) Veure fotos per usuari: a l'URL `fotos/usuari/`. Aquesta pàgina serà un formulari on hem d'escriure un nom d'usuari de la xarxa Instagram. En enviar el formulari ens durà a veure les fotos corresponents a un URL del tipus `fotos/usuari/nom_usuari` (per exemple `fotos/usuari/barackobama`)

3) Veure fotos per *tag*: a l'URL `fotos/tag/` hi haurà un formulari on introduirem un *tag*. En clicar al botó "veure" ens durà a un URL on es veuran les darreres fotos publicades amb aquesta etiqueta (per exemple `fotos/usuari/cat`).

El primer pas serà baixar el programari de CodeIgniter 3 del web oficial (http://www.codeigniter.com/). Copiarem els arxius al nostre servidor comprovant que en entrar amb el navegador a l'URL corresponent, per exemple localhost/code_igniter ens apareix el missatge *welcome to codeigniter*. En aquest moment podem dir que ja tenim CodeIgniter funcionant.

Com a punt de partida CI té un controlador, un model i una vista. Cadascun d'ells es troben al directori corresponent `controllers`, `models` o `views`. Comencem amb el controlador, l'únic existent, que el trobarem dins de la carpeta `/application/controllers` i s'anomena `welcome.php`.

Obrint el codi del controlador podrem comprovar la seva definició:

```
class Welcome extends CI_Controller {
 public function index()
 {
  $this->load->view('welcome_message');
 }
}
```

Tots els controladors deriven de la classe base `CI_Controller`. El controlador en aquest cas només té un mètode anomenat *index* que l'únic que fa és carregar una vista anomenada *welcome_message*.

L'URL que correspondria a aquest punt d'entrada seguint la convenció de CI seria `index.php/welcome/index`, on `welcome` és el nom del controlador i `index` el nom del mètode.

Però, aleshores, com és que apareix a l'arrel de la nostra aplicació? La resposta la trobarem a la carpeta `config`, on hi ha un arxiu anomenat `routes.php` on s'especifica que el controlador per defecte és *welcome*.

```
$route['default_controller'] = 'welcome';
```

CodeIgniter entén que si no especifiquem cap controlador farà servir aquest, i en general si no especifiquem cap mètode del controlador sempre farà servir el mètode anomenat `index`. Tots els controladors en general tenen un mètode `index` que és el que s'utilitza per defecte.

Farem una prova per entendre el funcionament dels URL i els controladors. Crearem un altre controlador anomenat "fotos" copiant el controlador *welcome*.

```
class Fotos extends CI_Controller {
 public function index()
 {
  $this->load->view('welcome_message');
 }
 public function test($num=0)
 {
  print "num és ".$num;
 }
}
```

En aquest cas, si visitem l'URL `index.php/fotos/test/20` comprovarem que apareix el número 20 a la pantalla. Veiem com a l'URL `fotos` correspon al nom del controlador, `test` al nom del mètode i `20` en aquest cas és el paràmetre que passem a aquest mètode (podem afegir tants paràmetres com vulguem).

Per eliminar `index.php` de l'URL l'únic que hem de fer és afegir un arxiu `.htaccess`, sempre que tinguem activat el mòdul `mod_rewrite` d'Apache (en cas contrari no serviria de res) amb el següent contingut:

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php/$1 [L]
```

D'aquesta manera l'URL anterior queda simplificat a `fotos/test/20`.

A l'exemple anterior el controlador mostra directament el paràmetre `$num`, però naturalment la manera correcta seria carregar una vista i passar-li aquest paràmetre. El controlador passa dades a la vista a través d'un segon paràmetre com mostrem a continuació:

```
public function test($num=0)
{
 //manera de passar els paràmetres a la vista
 $data=array(
 'num'=>$num,
 );
 $this->load->view('fotos',$data);
}
```

I a la carpeta *views* afegirem un arxiu `fotos.php` on a banda de la resta de l'HTML mostrarem el valor de `$num`.

```
<?php echo $num ?>
```

Per accedir a un full d'estils extern o altres recursos (JavaScript per exemple) podem crear un directori *assets* a l'arrel de CI i a la nostra plantilla afegir:

```
<link rel="stylesheet" href="<?php echo base_url();?>assets/css/style.css" type="text/css"
media="screen"/>
```

`base_url()` ens dóna l'URL de la instal·lació actual de CI, de manera que garantim que seguirà funcionant encara que canviem de localització. Però perquè funcioni aquesta instrucció cal activar-la a l'arxiu de configuració. Dins de *config* trobarem l'arxiu `autoload.php`. En aquest arxiu haurem de modificar la següent línia:

```
$autoload['helper'] = array('url');
```

`base_url()` és una de les funcions d'aquesta llibreria d'ajuda.

A continuació construirem el model. Cada model és un arxiu que resideix al directori `/models` i que defineix una classe derivada de CI_Model. Com ja sabem els models representen l'estructura de dades de la nostra aplicació.

En el nostre cas per tant definirem tres mètodes:

1) `get_user_id($username)`, que retornarà l'ID d'un usuari d'Instagram a partir del seu nom d'usuari,

2) `get_photos`, que retornarà les dades de les fotos d'un usuari a partir del seu ID, i

3) `get_photos_by_tag`, que retornarà les fotos que tenen una determinada etiqueta (*hashtag*).

Com podem observar al codi el model deriva de la classe base CI_Model.

```
class Instagram_model extends CI_Model {
```

Cada model tindrà una sèrie de mètodes que farem servir des dels controladors.

Al constructor trobem aquestes dues línies que corresponen al sistema de configuració de CI:

```
$this->config->load('instagram');
$this->instagram_key=$this->config->item('INSTAGRAM_KEY');
```

A la primera, CI carrega l'arxiu de configuració anomenat `instagram.php` al directori `/config`. A continuació s'assigna el valor de l'entrada `INSTAGRAM_KEY` d'aquest arxiu a la propietat del model.

Aquesta és la clau de l'API que es fa servir als mètodes del model per a fer les crides a Instagram.

A continuació veiem com hem construït el controlador Fotos:

```
class Fotos extends CI_Controller {
```

El controlador deriva de la classe base CI_Controller i té una sèrie de mètodes definits que corresponen als URL de la nostra aplicació. Així tenim:

1) índex: correspon a la ruta `/fotos`

2) test: `/fotos/test`

3) usuari: `/fotos/usuari/username`. Aquest mètode té un paràmetre que correspon al tercer segment de l'URL, de manera que si l'URL és per exemple `/fotos/usuari/barackobama` aquest serà el paràmetre que es passarà al mètode:

```
public function usuari($username=""){
```

Si afegim més paràmetres a la funció aquests serien els corresponents als segments de l'URL successius, per exemple:

```
public function foo($a,$b,$c){
```

correspondria a l'URL:

```
/controlador/foo/a/b/c
```

Normalment als controladors carreguem els models de la nostra aplicació i passem les dades del model a la vista. En el nostre cas, primer carreguem el model:

```
$this->load->model('instagram_model');
```

i després fem una crida a un mètode per obtenir les fotos de l'usuari:

```
$photos=$this->instagram_model->get_photos($user_id,8);
```

En realitat a CI els models es poden carregar al constructor o, si són necessaris, sovint fins i tot es poden carregar automàticament afegint una entrada a l'arxiu `/config/autoload.php` igual que hem fet amb la llibreria d'ajuda "url" anteriorment.

Finalment els mètodes del controlador carreguen la vista i li passen les dades a través d'un paràmetre:

```
$this->load->view('fotos_user', array('username'=>$username,'fotos'=>$photos) ) ;
```

Aquestes dades són les que podem fer servir a les vistes, per exemple en el cas de les fotos d'un usuari (vista `fotos_user.php`):

```
<?php foreach($fotos as $foto): ?>
<a href='<?php echo $foto->link?>'>
<img src='<?php echo $foto->images->standard_resolution->url?>'>
</a>
<?php endforeach; ?>
```

Per ampliar informació

Trobareu el codi d'aquest exemple sencer a https://github.com/danieljulia/backend_ci_instagram

4.2.2. Bases de dades a CI

Al nostre exemple el model no accedeix a la base de dades, però en el cas que tinguem una base de dades caldrà carregar la classe *database* al controlador com veiem a continuació:

```
$this->load->database();
```

Un cop carregada la classe i afegides les dades de connexió a l'arxiu `config/database.php`, podem fer operacions amb la base de dades al nostre model, com per exemple un *select*:

```
$query = $this->db->query('SELECT name FROM my_table LIMIT 1');

$row = $query->row();

echo $row->name;
```

o per exemple un *insert*:

```
$data = array(
        'title' => $title,
        'name' => $name,
        'date' => $date
);

$this->db->insert('mytable', $data);
```

Per ampliar informació

Sobre la classe *database* de CI: https://codeigniter.com/userguide3/database/index.html

Exercici

Es demana modificar l'exemple anterior, afegint els següents URL que corresponen a dos nous mètodes al controlador i al model:

```
fotos/follows/username
user/followed-by/username
```

Aquests URLs han de mostrar les imatges de perfil de tots els usuaris que segueixen o pels quals són seguits els usuaris amb el *username* corresponent.

Per ampliar informació

Documentació sobre l'API d'Instagram: https://www.instagram.com/developer/endpoints/

4.2.3. Extensions de CI

La simplicitat de CI implica que li manquen algunes funcionalitats que a vegades són importants. Aquestes funcionalitats es poden solucionar amb extensions externes.

Veiem algunes d'aquestes extensions:

1) Autenticació: és una de les necessitats més freqüents de qualsevol aplicació web. Existeixen diverses extensions, com per exemple Ion Auth (https://github.com/benedmunds/CodeIgniter-Ion-Auth), que ens permet definir diferents rols d'usuari.

Un cop configurada l'extensió podem verificar, per exemple, si l'usuari no està "loginat" i redirigir-lo a la pàgina de *login*, aquesta condició pot estar al constructor del controlador, de manera que caldrà que l'usuari estigui autentificat per qualsevol mètode del controlador.

```
if (!$this->ion_auth->logged_in()) {
 redirect('auth/login');
```

2) CRUD: aquesta funcionalitat, que pot estalviar molt temps de desenvolupament, la podem solucionar amb l'extensió Grocery CRUD (http://www.grocerycrud.com/). El que ens permet és que a partir de l'estructura de la base de dades, taules amb camps i relacions, podem crear fàcilment una eina de gestió d'aquestes dades CRUD, que ens permeti *Create* (crear), *Read* (llegir), *Update* (actualitzar) i *Delete* (esborrar) fàcilment de manera semblant com ho fem amb l'escriptori de WordPress però totalment personalitzat.

La combinació de les dues anteriors llibreries permet tenir un *framework* més complet, amb autentificació i una eina de CRUD.

Podeu descarregar CodeIgniter amb aquestes dues llibreries a GitHub (https://github.com/danieljulia/CodeIgniter-3-Grocery-Crud-Ion-Auth).

Recursos addicionals CodeIgniter

Codeigniter REST server. Implementació simple d'un servidor REST amb CI: https://github.com/chriskacerguis/codeigniter-restserver

Tutorial oficial de CodeIgniter: http://www.codeigniter.com/user_guide/tutorial/index.html

Tutorials en vídeo a Youtube: https://www.youtube.com/playlist?list=PLUpnKy5Si8zDouvZiUMHwSSyVrowJmH22

4.3. Laravel

Laravel és un altre *framework* de desenvolupament, amb força més funcionalitats que CodeIgniter. Laravel, igual que Drupal 8, fa servir alguns dels components de Symfony i està basat en les noves capacitats de PHP 5, a diferència de CI.

A més, Laravel incorpora funcionalitats importants que no té CI com són l'autenticació, el *routing*, les sessions o un sistema de memòria cau, però sempre mantenint la filosofia de senzillesa de CI.

Per ampliar informació

Introducció sobre Laravel i comparativa amb altres *frameworks*: https://vimeo.com/45417023

Documentació oficial: http://symfony.com/projects/laravel

Quickstart: http://laravel.com/docs/4.2/quick

Laracasts, Tutorials en vídeo: https://laracasts.com/

Build Larabook From Scratch: https://laracasts.com/series/build-a-laravel-app-from-scratch

4.4. Symfony

Symfony és un entorn de desenvolupament (*framework*) i al mateix temps també una sèrie de components (llibreries).

Per ampliar informació

Web oficial de Symfony: http://symfony.com/

Documentació: http://symfony.com/doc/current/index.html

Introducció: http://symfony.com/at-a-glance

Presentació sobre Symfony: http://www.slideshare.net/kriswallsmith/symfony2-tokyo

Helloworld: http://www.whitewashing.de/2014/04/24/symfony_hello_world.html

Tutorial pas a pas d'una aplicació en Symfony: http://www.ens.ro/2012/03/21/jobeet-tutorial-with-symfony2/

4.5. JavaScript al servidor, MEAN (MongoDB, Express, Angular, NodeJS)

4.5.1. MEAN

La plataforma més utilitzada actualment a les infraestructures de servidors a internet és la que anomenem LAMP, inicials que provenen del sistema operatiu Linux, el servidor web Apache, la base de dades MySQL i el llenguatge de servidor PHP. Tots ells són components de programari obert que han facilitat durant anys la creació de gran quantitat de recursos i estan darrere de la majoria de webs actuals a internet.

Però gràcies a node.js cada vegada es parla més de MEAN (inicials que provenen de la base de dades MongoDB, el *framework* de servidor ExpressJS, el *framework* client AngularJS i la plataforma de servidor NodeJS).

Per ampliar informació

Mean fullstack javascript framework: http://mean.io/#!/

Tots ells funcionen sobre la filosofia JavaScript i el format d'arxius JSON. Veiem a continuació cadascun d'aquests components.

4.5.2. MongoDB

MongoDB és una base de dades NoSQL. MongoDB emmagatzema les dades en documents amb format JSON, cosa que el fa ideal per a treballar en entorns JavaScript. Podem imaginar una base de dades en MongoDB com un immens arxiu JSON sobre el qual podem fer cerques o actualitzacions.

Un cop instal·lat podem accedir a la base de dades a través de la línia de comandes. A cada base de dades podem tenir diferents *collections* que d'alguna manera equivalen al concepte de taula a una base de dades relacional, tot i que els camps de cada document poden ser diferents.

Veiem com podem fer a través de la línia de comandes les quatre operacions bàsiques CRUD (*Create*, *Read*, *Update* i *Delete*):

1) *Create*: crear dos documents a la col·lecció *users* (si no existeix es crea automàticament)

```
db.users.save({ name: 'Anna',edat: 35 },{ name: 'Joan' });
```

2) *Read*

```
db.users.find({ name: 'Anna' });
```

3) *Update*

```
db.users.update({ name: 'Joan' }, { name: 'Pere' });
```

4) *Delete*

```
db.users.remove({ name: 'Anna' });
```

Una altra diferència amb les bases de dades relacionals és que no hi ha relacions i per tant és la mateixa aplicació que haurà d'implementar algun sistema per a poder relacionar diferents col·leccions.

Per ampliar informació

Modes relacionals a MongoDB: https://docs.mongodb.org/v3.0/applications/data-models-relationships/

Web oficial de MongoDB: https://docs.mongodb.org/

Documentació oficial de MongoDB: https://docs.mongodb.org/manual/

4.5.3. Node.js

El llenguatge JavaScript ha estat present pràcticament des dels inicis del web a la banda del client, als navegadors, però no ha estat fins als darrers anys que també s'ha incorporat a la banda del servidor.

Node.js és una plataforma que fa servir un intèrpret de JavaScript basat en el V8 Engine (la mateixa implementació que s'utilitza al navegador Google Chrome). Node.js està pensat per a desenvolupar aplicacions intensives que funcionen a la xarxa i amb un gran flux de dades. Tot i les seves peculiaritats específiques el llenguatge amb el qual es programa aquesta plataforma és JavaScript estàndard.

Per primera vegada a la història del desenvolupament web podem tenir perfils *full-stack* que fan servir el mateix llenguatge de programació tant per a la part del client com per a la del servidor.

Tot i estar encara en desenvolupament Node.js ha experimentat un creixement espectacular. Ateses les seves característiques és un llenguatge molt apropiat per a desenvolupar aplicacions web que poden funcionar en temps real, a diferència de l'estratègia asíncrona dels llenguatges clàssics com PHP.

Un cop instal·lat Node.js al nostre ordinador o servidor podem escriure programes en JavaScript i executar-los a la línia de comandes.

Imaginem un exemple ben senzill:

```
var a=2;
console.log(a*2);
```

Per executar-lo haurem d'escriure a la línia de comandes:

```
node test.js
```

I veurem el resultat. De fet és molt semblant al que podríem fer a la consola de JavaScript d'un navegador web com Chrome.

Però en realitat l'exemple anterior no ens serveix a un entorn web ja que, a diferència de PHP, que funciona com una extensió del servidor web Apache, aquí és Node que ha de funcionar també com a servidor web. A continuació veiem un exemple on es mostra el missatge *hello world* quan es fa una petició al servidor local al port 3000.

```
var http = require('http');
http.createServer(function (req, res) {
 res.writeHead(200, {'Content-Type': 'text/plain'});
 res.end('Hello World!');
}).listen(3000, '127.0.0.1');
```

En realitat el programa està fent moltes més coses a banda de mostrar el missatge: crea un servidor web que "escolta" a un port específic i dóna com a resposta el missatge *hello world*. Per tant, veiem que no necessitem cap servidor web com Apache, i és el mateix Node que s'encarrega d'aquesta tasca.

Node.js és una plataforma orientada a esdeveniments: en el cas anterior estem fent servir, per exemple, l'esdeveniment *listen*. Per aquest motiu és un llenguatge asíncron. Aquest fet també és el que fa que sigui realment ràpid i escalable comparat amb altres llenguatges clàssics "seqüencials" com PHP.

Per ampliar informació

Instal·lació de Node.js: https://nodejs.org/en/download/

Web oficial de Node.js: https://nodejs.org/en/

Introducció al model d'esdeveniments de Node.js: http://www.genbetadev.com/frameworks/como-funciona-node-js

4.5.4. Npm

Una de les utilitats que inclou nodejs és npm (*node package manager*). Es tracta d'una utilitat que permet reutilitzar codi entre els usuaris de Node. Aquest codi s'organitza en forma de *packages*. Cada *package* és un directori amb una sèrie

d'arxius que resolen un problema concret. Amb npm és molt fàcil descarregar aquests *packages* o actualitzar-los quan hi ha una nova versió, o fins i tot pujar el nostre propi *package* per a ser reutilitzat per la resta de la comunitat. Actualment hi ha més de 200.000 *packages* al directori www.npmjs.com/.

Els *packages* es poden instal·lar localment (per utilitzar al projecte actual):

```
npm install <package>
```

o globalment (per tots els projectes de Node):

```
npm install -g <package>
```

Els projectes Node.js fan servir extensivament *packages*, ja que és una de les principals maneres de poder reaprofitar codi.

Per ampliar informació

Directori de *packages*: https://www.npmjs.com/

Tutorials de Node: https://docs.npmjs.com/getting-started/installing-node

Què és npm: https://docs.npmjs.com/getting-started/what-is-npm

4.5.5. Express

Express és un *framework* MVC basat en Node.js. De fet Express és un *package* i per tant s'instal·la com un *package* qualsevol:

```
npm install express
```

Un cop instal·lat el podem incloure al nostre codi amb *require*. Veiem un exemple de *Hello World* amb Express, a diferència de l'anterior que era implementat només amb Node.js:

```
var express = require('express');
var app = express();

app.get('/', function (req, res) {
  res.send('Hello World!');
});

var server = app.listen(3000, function () {
  var host = server.address().address;
  var port = server.address().port;

  console.log('Example app listening at http://%s:%s', host, port);
});
```

Per executar el codi anterior (suposant que l'arxiu es digui app.js) caldrà escriure a la línia de comandes:

```
node app.js
```

Per veure el resultat cal obrir a un navegador l'adreça `http://localhost:3000/`.

El *package* `express-generator` ens permet crear l'esquelet de la nostra aplicació fàcilment. Per instal·lar-lo:

```
npm install express-generator -g
```

I per crear l'esquelet:

```
express myapp
```

4.5.6. Loopback

Loopback és un *framework* basat en Node, com Express, però enfocat a crear API.

S'instal·la com qualsevol *package* de Node:

```
npm install -g strongloop
```

Un cop instal·lat podem crear fàcilment el nostre model de dades amb les propietats que volem i Loopback s'encarrega de la resta.

Per ampliar informació

Documentació oficial express: http://expressjs.com/

Per ampliar informació

http://loopback.io/getting-started/

4.6. Xarxes socials de codi

Seguint el principi de "no reinventar la roda", de la mateixa manera que podem aprofitar components de CMS per a crear la nostra aplicació web, els CMS també s'han beneficiat d'altres components PHP que s'han incorporat els darrers anys dins del creixent repositori de possibilitats.

Això és en part gràcies a l'ús cada vegada més freqüent d'eines que faciliten la compartició de codi, com per exemple les eines de control de versions com Git, o les comunitats de codi com GitHub.

Els sistemes de control de versions com Git, Mercurial o SVN existeixen des de fa molt temps, però el que ha estat realment important darrerament és el creixement de xarxes socials com GitHub.

GitHub s'ha convertit en la xarxa social per excel·lència de la compartició de codi, igual que Facebook ho és de les relacions socials.

El codi dels projectes públics de GitHub es pot consultar lliurement, veure totes les contribucions que s'han fet amb el temps o fer modificacions (*forks*).

Igual que a Facebook o Twitter a GitHub un usuari pot seguir-ne a un altre i pot marcar com a favorits projectes.

A més de GitHub hi ha alternatives, entre altres les següents:

1) Bitbucket: té l'avantatge que permet tenir més repositoris privats.

2) Phabricator: és una plataforma lliure i es pot instal·lar a qualsevol servidor.

4.7. Gestors de paquets

És evident que, en general, el fet de compartir el codi i reutilitzar components facilita molt el desenvolupament en l'àmbit global, per una banda fa el codi molt més modular i per un altre permet reutilitzar coneixements entre diferents projectes.

Hi ha eines que ens permeten gestionar els projectes i controlar automàticament les dependències dels components i les llibreries que fem servir i les seves versions.

Exemple

Imaginem un cas concret, a un projecte hem de fer servir Twitter Bootstrap com a eina de composició del *frontend*. Si descarreguem manualment aquesta llibreria hem de baixar també la llibreria jQuery i altres llibreries associades que puguin fer falta. Amb una eina de control de dependències tot això es pot fer automàticament.

Gestors de paquets més importants existents actualment:

1) npm, gestor de paquets per a Node.js.

2) Bower per a projectes *frontend*.

3) Composer, dirigit a projectes PHP.

En general el funcionament d'aquestes eines és el següent: es declara en un arxiu (normalment en format JSON) quins són els components que volem fer servir al nostre projecte. A partir d'aquest moment l'eina s'encarregarà de descarregar-los i gestionar les dependències que hi pugui haver.

Veiem un exemple amb Bower. Primer cal instal·lar-lo: si ja tenim Node.js instal·lat, que inclou el seu propi gestor de paquets, npm:

```
npm install -g bower
```

Primer hem de crear l'arxiu de configuració de Bower, `bowerrc.json`, on especifiquem on s'han de desar els components:

```
{
```

```
    "directory" : "web/vendor/"
}
```

A continuació crearem un altre arxiu on escriurem les dependències del nostre projecte. Aquest arxiu es diu `bower.json`. En el nostre cas només especifiquem que farem servir Bootstrap:

```
{
    "name": "MyProject",
    "dependencies": {
        "bootstrap": "*"
    }
}
```

Finalment "instal·larem" les dependències amb la següent instrucció:

```
bower install
```

Si observem el directori *web/vendor* veurem que s'han baixat tots els arxius de la llibreria Bootstrap i també JQuery (ja que Bootstrap depèn d'ells).

5. Estudis de cas

Com a exemples de desenvolupament analitzarem dos estudis de cas i veurem per a cadascun d'ells com podem adaptar-los a tres de les plataformes que hem vist, WordPress, Drupal i CodeIgniter.

5.1. Cas 1. Catàleg educatiu interactiu

Aquest primer exemple d'aplicació consistirà en un catàleg educatiu interactiu (la temàtica del contingut podria ser per exemple sobre ornitologia, arbres, plantes, micologia, etc.).

L'objectiu és desenvolupar una aplicació que serveixi per a interactuar amb aquest catàleg, de manera "lineal" però també a través d'un joc interactiu que permeti als usuaris aprendre jugant.

5.1.1. Definició dels requeriments de l'aplicació

Per una banda cal crear una eina de gestió que permeti donar d'alta els continguts del catàleg. Bàsicament es tracta d'una col·lecció d'ítems.

Cada ítem del catàleg tindrà els següents camps :

1) Nom

2) Descripció

3) Foto

4) Categories

5) Etiquetes

Per a desenvolupar el *frontend* caldrà una API que ens permeti fer les següents consultes:

1) `/GET items`: obté el llistat total d'ítems amb tots els seus camps.

2) `/GET id_categoria/items`: obté el llistat d'ítems d'aquesta categoria.

3) `/GET id_tag/items`: obté el llistat d'ítems d'aquesta etiqueta (*tag*).

4) `/GET categoria`: obté el llistat de totes les categories i els seus ID.

5) `/GET tag`: obté el llistat de totes les categories i els seus ID.

Jocs

A la part del *frontend* caldrà desenvolupar dos jocs per interactuar amb el catàleg:

1) Endevina el nom l'ítem: apareix la foto d'un ítem i tres noms possibles de tota la base de dades. S'ha de triar l'opció correcta. Si s'encerta es suma un punt. Es continua fins a acabar tots els continguts del catàleg.

2) Endevina la foto de l'ítem: la mateixa idea però apareixen tres fotos i el nom d'un dels ítems. S'ha de triar la foto que correspon al nom.

El *frontend* ha d'estar optimitzat per a mòbils i tauletes.

5.1.2. Anàlisi de pros i contres per a la solució en WordPress, Drupal i CodeIgniter

WordPress

En el cas de WordPress tenim un avantatge, ja que l'estructura de continguts de les entrades (*posts*) correspon a l'estructura d'un ítem tal com es demana a l'enunciat. Els *posts* de WP tenen un títol, una descripció, una imatge destacada, categories i etiquetes associades. Per tant amb WP ja tenim l'eina per a gestionar els continguts i només ens cal implementar l'API REST de consulta de dades. Per aquest propòsit instal·larem el *plugin* REST API, si encara no el tenim activat.

Drupal

En el cas de Drupal podem definir fàcilment un tipus de contingut *item* amb els camps requerits. Després caldrà activar els mòduls RESTful Web Services i Serialization, configurar els permisos per a poder fer accessos GET amb l'usuari anònim i finalment crear vistes per a cadascun dels punts d'entrada que es demanen a l'enunciat.

CodeIgniter

En el cas de CI els passos que haurem de seguir són els següents:

1) Instal·lar CI amb Grocery Crud.

2) Crear la base de dades i les taules corresponents.

3) Definir els controladors amb Grocery CRUD per poder administrar els continguts (ítems, categories i etiquetes).

4) Definir un controlador REST que faci les peticions a la base de dades i les converteixi a format JSON.

Tots aquests passos són relativament senzills i els podem aconseguir ràpidament amb CI, almenys per a una primera versió.

Per ampliar informació

Primera versió de la implementació anterior: https://github.com/danieljulia/backend_cas_us_1_ci

Una llibreria per a fer interfícies REST amb CI: https://github.com/chriskacerguis/codeigniter-restserver

L'enfocament per a cadascuna d'aquestes plataformes és totalment diferent. Així veiem que en el cas de WordPress la implementació és relativament simple però ens cal sovint fer modificacions al codi. El cas de Drupal és complex però es pot resoldre només amb la configuració del seu panell de control i finalment a CodeIgniter tenim flexibilitat total però tot ho hem de fer a partir del codi.

5.2. Cas 2. Aplicació mòbil

Com a segon exemple d'aplicació crearem una aplicació mòbil híbrida que està pensada per a ser encapsulada com a aplicació nativa per a les plataformes iOS i Android amb un sistema com PhoneGap.

L'aplicació serveix per a consultar la informació d'un portal de notícies. En aquest moment ja funciona i el codi està disponible a GitHub, però s'alimenta d'arxius JSON estàtics.

L'exercici consisteix en crear el *backend* per gestionar els continguts en els sistemes que hem vist i fer les modificacions necessàries perquè el contingut de l'aplicació s'alimenti del CMS o *framework* corresponent.

L'API, per tant, haurà de tenir dos punts d'entrada:

1)`/GET news`: ara simulada per un arxiu news.json

2)`/GET new/id`: per demanar el contingut d'una notícia en concret ara simulat amb post.json

Per ampliar informació

Codi font de l'exemple de partida: https://github.com/danieljulia/wp-rest-api-demo

Web de PhoneGap: http://phonegap.com/

Tutorial en el qual està basat l'exemple: https://apppresser.com/using-wordpress-rest-api-mobile-app/